

===== FILE: .claude/settings.local.json =====

```
1: {
2:   "permissions": {
3:     "allow": [
4:       "Read(//c/Users/qisaihao/给ai的图片/**)",
5:       "Bash(cat:*)",
6:       "Bash(npm run dev:*)",
7:       "Bash(git clone:*)",
8:       "Bash(git checkout:*)",
9:       "Bash(cp:*)",
10:      "Bash(rm:*)",
11:      "Read(//c/Users/86199/给ai的图片/**)",
12:      "Bash(node:*)",
13:      "Bash(npx @cloudbase/cli functions:deploy:*)",
14:      "Bash(tcb functions:deploy:*)",
15:      "Bash(tcb:*)",
16:      "Bash(dir:*)",
17:      "Bash(awk:*)",
18:      "Bash(npm run)",
19:      "Bash(npx @dcloudio/uvm)",
20:      "Bash(npm run dev:app-plus:*)",
21:      "Bash(powershell:*)",
22:      "Bash(clean_logs.bat:*)",
23:      "WebFetch(domain:doc.dcloud.net.cn)",
24:      "WebSearch",
25:      "Bash(mkdir:*)",
26:      "Bash(npm install)",
27:      "Bash(tcloud fn run:*)",
28:      "Bash(GITHUB_REDIRECT_URI=\"https://cloud1-5gb0pbyl400845f5-1378788263.ap-shanghai.app.tcloudbase.com/github-auth\"...",
29:      "Bash(GITHUB_REDIRECT_URI=\"https://cloud1-5gb0pbyl400845f5-1378788263.ap-shanghai.app.tcloudbase.com/github-auth\"...",
30:      "Bash(while read dir)",
31:      "Bash(do if ! grep -q \"pages/$dir/\" pages.json)",
32:      "Bash(then echo \"未在pages.json中注册: $dir\")",
33:      "Bash(fi:*)",
34:      "Bash(done)",
35:      "Bash(find:*)",
36:      "Bash(npm run dev:h5:*)",
37:      "Bash(for:*)",
38:      "Bash(do echo:*)"
39:    ],
40:    "deny": [],
41:    "ask": []
42:  }
43: }
```

===== FILE: .hbuilderx/launch.json =====

```
1: {
2:   "version": "1.0",
3:   "configurations": [
4:     {
```

```

5:         "customPlaygroundType" : "local",
6:         "playground" : "standard",
7:         "type" : "uni-app:app-android"
8:     },
9:     {
10:        "app" : {
11:            "launchtype" : "remote"
12:        },
13:        "default" : {
14:            "launchtype" : "local"
15:        },
16:        "h5" : {
17:            "launchtype" : "remote"
18:        },
19:        "provider" : "aliyun",
20:        "type" : "uniCloud"
21:    }
22: ]
23: }

```

===== FILE: .vscode/settings.json =====

```

1: {
2:     "git.ignoreLimitWarning": true
3: }

```

===== FILE: 1.css =====

===== FILE: App.vue =====

```

1: <script>
2: // #ifdef APP-PLUS
3: // import checkUpdate from '@uni_modules/uni-upgrade-center-app/utils/check-update';
4: import { checkAndUpdate } from '@utils/hotUpdate.js';
5: // #endif
6: import fontManager from '@utils/fontManager.js';
7:
8: export default {
9:     // 【重构】1. 将所有全局数据放入 data 函数中，这是 Vue 的标准做法
10:    data() {
11:        return {
12:            // globalData 对象将在这里被 Vue 响应式地管理
13:            globalData: {
14:                userInfo: null,
15:                openid: null
16:            // 您可以保留其他全局变量，但 env ID 已在 main.js 中配置，这里不再需要
17:            }
18:        };
19:    },
20:
21:    // 【重构】2. onLaunch 是 Vue 的生命周期函数，保持不变
22:    onLaunch: function (options) {

```

```

23: // #ifdef APP-PLUS
24: // 处理 URL Scheme 启动 (GitHub OAuth 回调)
25: const args = plus.runtime.arguments;
26: console.log('□ [App] onLaunch 启动参数:', args);
27: if (args && args.includes('github-callback')) {
28:     console.log('□ [URL Scheme] 检测到 GitHub 回调 (onLaunch) ');
29:     this.handleUrlScheme({ path: args });
30: }
31: // #endif
32:
33: // 清理缓存是好习惯, 予以保留
34: uni.removeStorageSync('cachedPostList');
35:
36: // #ifdef APP-PLUS
37: console.log('□ [热更新] 当前为 APP-PLUS 环境, 准备获取版本信息...');
38: try {
39:     const systemInfo = uni.getSystemInfoSync();
40:     const appId = systemInfo.appId;
41:     console.log('□ [热更新] App ID:', appId);
42:
43:     plus.runtime.getProperty(plus.runtime.appid, function(widgetInfo) {
44:         console.log('□ [热更新] plus.runtime.getProperty 回调已执行');
45:         if (widgetInfo) {
46:             // 使用 widgetInfo.versionCode 获取版本号
47:             const appVersion = widgetInfo.versionCode || 0;
48:             const wgtVersion = widgetInfo.version || '';
49:             console.log('□ [热更新] App 版本号 (versionCode):', appVersion);
50:             console.log('□ [热更新] WGT 资源包版本号 (wgtVersion):', wgtVersion);
51:             console.log('□ [热更新] 当前完整版本信息:', {
52:                 appId: appId,
53:                 appVersion: appVersion,
54:                 wgtVersion: wgtVersion
55:             });
56:             // 使用自定义热更新逻辑
57:             try {
58:                 checkAndUpdate({ silent: true, showConfirm: true }).then((result) => {
59:                     console.log('□ [热更新] 检查完成:', result);
60:                 }).catch((err) => {
61:                     // 区分不同类型的错误, 提供更友好的提示
62:                     const errorMsg = err?.errMsg || err?.message || String(err);
63:                     if (errorMsg.includes('resource exhausted') || errorMsg.includes('ResourceExhausted') || (err...
64:                         console.warn('□□ [热更新] 检查失败: 云服务资源已耗尽, 请稍后重试');
65:                     } else {
66:                         console.warn('□□ [热更新] 检查失败 (已忽略):', err);
67:                     }
68:                 });
69:             } catch (e) {
70:                 console.warn('□□ [热更新] 调用异常 (已忽略):', e);
71:             }
72:         } else {

```

```

73:         console.error('❌ [热更新] 获取 WGT 版本信息失败, 请检查 manifest.json 配置或运行环境。');
74:     }
75: });
76: } catch (error) {
77:     console.warn('❌❌ [热更新] 读取版本信息失败 (已忽略):', error);
78: }
79: // #endif
80:
81: // #ifdef H5
82: console.log('当前为 H5 环境, 跳过热更新检查');
83: // #endif
84:
85: // 【性能优化】立即标记登录流程已开始, 不阻塞后续操作
86: this.globalData._loginProcessStarted = true;
87:
88: // 【字体预加载】提前加载默认字体, 避免第一次生成卡片时字体未就绪
89: fontManager.preloadCommonFonts(['汇文明朝']).then(results => {
90:     console.log('【App】默认字体预加载完成:', results);
91: }).catch(err => {
92:     console.warn('【App】默认字体预加载失败:', err);
93: });
94:
95: // 【性能优化】使用 nextTick 延迟执行非关键任务, 让页面先渲染
96: this.$nextTick(() => {
97:     // 执行登录流程 (异步, 不阻塞UI)
98:     this.loginAndCheckUser();
99: });
100: },
101:
102: // 【新增】onShow 生命周期: 处理 App 从后台唤醒的情况
103: onShow: function(options) {
104:     console.log('【App】onShow 触发');
105:
106:     // #ifdef APP-PLUS
107:     // 获取启动参数 (URL Scheme)
108:     const args = plus.runtime.arguments;
109:     console.log('【URL Scheme】启动参数:', args);
110:
111:     // 如果有 URL Scheme 参数, 则处理
112:     if (args && args.includes('github-callback')) {
113:         console.log('【URL Scheme】检测到 GitHub 回调');
114:         this.handleUrlScheme({ path: args });
115:     }
116:
117:     // 监听新的 intent 事件 (Android 特有)
118:     // 当 App 已在后台运行时, 通过 URL Scheme 唤起会触发此事件
119:     // 注意: 避免重复添加监听器
120:     if (!this._newintentListenerAdded) {
121:         plus.globalEvent.addEventListener('newintent', (e) => {
122:             console.log('【newintent】收到新的 intent 事件');

```

[illegible]

```

173:         uni.setStorageSync('github_temp_data', githubData);
174:         console.log(' [GitHub ] GitHub 数据已保存:', githubData);
175:     } catch (e) {
176:         console.error(' [GitHub ] 解析 GitHub 数据失败:', e);
177:     }
178:
179:     uni.showToast({
180:         title: '欢迎! 请完成注册',
181:         icon: 'none',
182:         duration: 2000
183:     });
184:
185:     // 跳转到注册页面
186:     setTimeout(() => {
187:         uni.reLaunch({
188:             url: `/pages/register/register?fromGithub=true&githubData=${encodeURIComponent(data)}`
189:         });
190:     }, 500);
191:
192: } else if (type === 'login') {
193:     // 已注册用户登录: 保存用户信息并跳转
194:     console.log(' [GitHub ] 已注册用户, 处理登录');
195:
196:     try {
197:         const loginData = JSON.parse(data);
198:         const userInfo = loginData.user;
199:
200:         // 更新全局用户信息
201:         this.globalData.userInfo = userInfo;
202:         this.globalData.openid = userInfo._openid || userInfo.openid;
203:
204:         // 更新 getApp().globalData
205:         const appInstance = getApp();
206:         if (appInstance) {
207:             appInstance.globalData = appInstance.globalData || {};
208:             appInstance.globalData.userInfo = userInfo;
209:             appInstance.globalData.openid = userInfo._openid || userInfo.openid;
210:             appInstance.globalData._loginProcessCompleted = true;
211:         }
212:
213:         // 保存到本地存储
214:         uni.setStorageSync('userInfo', userInfo);
215:         uni.setStorageSync('github_access_token', loginData.accessToken);
216:
217:         console.log(' [GitHub ] 用户信息已保存:', userInfo);
218:
219:         uni.showToast({
220:             title: '登录成功!',
221:             icon: 'success',
222:             duration: 2000

```

```

223:         });
224:
225:         // 跳转到诗歌广场
226:         setTimeout(() => {
227:             uni.reLaunch({
228:                 url: '/pages/poem-square/poem-square'
229:             });
230:         }, 500);
231:
232:     } catch (e) {
233:         console.error(' [GitHub ] 处理登录数据失败:', e);
234:         uni.showToast({
235:             title: '登录失败, 请重试',
236:             icon: 'none'
237:         });
238:     }
239: } else if (type === 'error') {
240:     // 错误处理: 显示错误信息
241:     console.log(' [GitHub ] 收到错误回调');
242:
243:     try {
244:         const errorInfo = JSON.parse(data);
245:         const errorMessage = errorInfo.message || 'GitHub登录失败, 请重试';
246:
247:         console.error(' [GitHub ] 错误信息:', errorMessage);
248:
249:         uni.showModal({
250:             title: 'GitHub 登录失败',
251:             content: errorMessage,
252:             showCancel: false,
253:             confirmText: '知道了',
254:             success: () => {
255:                 // 返回到登录页面
256:                 uni.reLaunch({
257:                     url: '/pages/login/login'
258:                 });
259:             }
260:         });
261:
262:     } catch (e) {
263:         console.error(' [GitHub ] 解析错误数据失败:', e);
264:         uni.showToast({
265:             title: 'GitHub登录失败',
266:             icon: 'none',
267:             duration: 2000
268:         });
269:
270:         // 返回到登录页面
271:         setTimeout(() => {
272:             uni.reLaunch({

```

```

273:                 url: '/pages/login/login'
274:             });
275:         }, 2000);
276:     }
277: }
278: } catch (error) {
279:     console.error(' [URL Scheme] 解析失败:', error);
280:     uni.showToast({
281:         title: '授权失败, 请重试',
282:         duration: 2000
283:     });
284: }
285: }
286: }
287: // #endif
288: },
289:
290: /**
291:  * 获取当前页面路径
292:  */
293: getCurrentPagePath() {
294:     try {
295:         const pages = getCurrentPages();
296:         const currentPage = pages[pages.length - 1];
297:         if (currentPage) {
298:             return currentPage.route || currentPage.$page?.fullPath || '';
299:         }
300:     } catch (e) {
301:         console.warn('获取当前页面路径失败:', e);
302:     }
303:     return '';
304: },
305:
306: /**
307:  * 检查是否是允许的页面 (splash、login 或 register)
308:  */
309: isAllowedPageForUnauthenticated() {
310:     const currentPath = this.getCurrentPagePath();
311:     // 允许 splash、login 和 register 页面在没有登录时访问
312:     return currentPath.includes('splash') || currentPath.includes('login') || currentPath.includes('register');
313: },
314:
315: // 【重构 & 修正】4. 使用 async/await 重写整个登录流程, 代码更清晰
316: async loginAndCheckUser() {
317:     // 检查 $tcb 实例是否存在
318:     if (!this.$tcb) {
319:         console.error('致命错误: this.$tcb 未定义! 请检查 main.js 的初始化代码是否执行! ');
320:         return; // 中断执行
321:     }
322:

```



```

323: // 步骤一：检查本地缓存，但需要验证云端账户
324: const cachedUserInfo = uni.getStorageSync('userInfo');
325: if (cachedUserInfo && cachedUserInfo._openid) {
326:   try {
327:     // 先进行匿名认证
328:     const currentUser = this.$tcb.auth().currentUser;
329:     if (!currentUser) {
330:       await this.$tcb.auth().signInAnonymously();
331:     }
332:
333:     // 调用云函数验证用户是否存在
334:     const verifyRes = await this.$tcb.callFunction({
335:       name: 'getUserProfile',
336:       data: { userId: cachedUserInfo._openid }
337:     });
338:
339:     if (verifyRes.result && verifyRes.result.success && verifyRes.result.userInfo) {
340:       // 使用云端返回的最新用户信息
341:       const latestUserInfo = verifyRes.result.userInfo;
342:
343:       // 同时更新 this.globalData 和 getApp().globalData
344:       this.globalData.userInfo = latestUserInfo;
345:       this.globalData.openid = latestUserInfo._openid;
346:
347:       // 确保 getApp().globalData 也被正确设置
348:       const appInstance = getApp();
349:       if (appInstance) {
350:         appInstance.globalData = appInstance.globalData || {};
351:         appInstance.globalData.userInfo = latestUserInfo;
352:         appInstance.globalData.openid = latestUserInfo._openid;
353:         console.log('☐ [登录流程] getApp().globalData 已更新:', appInstance.globalData);
354:       } else {
355:       }
356:
357:       // 更新本地缓存为最新的用户信息
358:       uni.setStorageSync('userInfo', latestUserInfo);
359:
360:       // 标记登录流程已完成
361:       this.globalData._loginProcessCompleted = true;
362:       if (appInstance) {
363:         appInstance.globalData._loginProcessCompleted = true;
364:       }
365:
366:       console.log('☐ [登录流程] 缓存验证成功，用户已登录');
367:       return; // 登录成功，结束流程
368:     } else {
369:       console.log('☐☐ [登录流程] 云端验证失败，用户账户不存在，将重新注册');
370:       // 清除无效的缓存
371:       uni.removeStorageSync('userInfo');
372:       // 继续执行注册流程

```

```

373:     }
374:   } catch (error) {
375:     console.error('☐ [登录流程] 云端验证失败:', error);
376:     console.log('☐☐ [登录流程] 验证失败, 将重新注册');
377:     // 清除可能无效的缓存
378:     uni.removeStorageSync('userInfo');
379:     // 继续执行注册流程
380:   }
381: }
382:
383: // 步骤二: 缓存未命中, 执行完整的云端登录
384: // 【关键修改】如果当前页面不是 splash 或 login, 且没有缓存, 则重定向到 splash
385: if (!this.isAllowedPageForUnauthenticated()) {
386:   console.log('☐☐ [登录流程] 检测到直接访问非登录页面, 且无登录缓存, 重定向到开屏页面');
387:
388:   // 【关键修复】在重定向之前, 确保 TCB 匿名认证已完成
389:   try {
390:     const currentUser = this.$tcb.auth().currentUser;
391:     if (!currentUser) {
392:       console.log('☐ [登录流程] 重定向前, 确保匿名认证完成...');
393:       await this.$tcb.auth().signInAnonymously();
394:       console.log('☐ [登录流程] 匿名认证已完成');
395:     }
396:   } catch (authError) {
397:     console.warn('☐☐ [登录流程] 匿名认证失败 (可能已在 main.js 中完成):', authError);
398:   }
399:
400:   // 延迟一小段时间, 确保页面加载完成
401:   setTimeout(() => {
402:     uni.reLaunch({
403:       url: '/pages/splash/splash'
404:     });
405:   }, 100);
406:   return; // 中断登录流程, 等待在 splash 页面重新执行
407: }
408:
409: console.log('☐ [登录流程] 缓存未命中, 开始执行云端登录...');
410:
411: try {
412:   // 检查是否已经登录, 避免重复登录
413:   const currentUser = this.$tcb.auth().currentUser;
414:   if (!currentUser) {
415:     console.log('☐ [认证] 尝试匿名登录...');
416:     const authResult = await this.$tcb.auth().signInAnonymously();
417:     console.log('☐ [认证] 匿名登录成功:', authResult);
418:   } else {
419:     console.log('☐ [认证] 用户已登录, 跳过匿名登录');
420:   }
421:
422:   // 【修正】调用 this.$tcb, 而不是 uniCloud!

```

```

423:     const loginRes = await this.$tcb.callFunction({
424:       name: 'login' // 调用您在 TCB 中的 login 云函数
425:     });
426:
427:     // 详细打印login云函数的返回数据，帮助调试
428:     console.log('☐ [调试] login云函数完整返回数据:', loginRes);
429:     console.log('☐ [调试] loginRes.result:', loginRes.result);
430:     console.log('☐ [调试] loginRes.openid:', loginRes.openid);
431:
432:     // 尝试多种方式获取openid
433:     let openid = null;
434:     if (loginRes.result && loginRes.result.openid) {
435:       openid = loginRes.result.openid;
436:       console.log('☐ [调试] 从result.openid获取到openid:', openid);
437:     } else if (loginRes.openid) {
438:       openid = loginRes.openid;
439:       console.log('☐ [调试] 从根级别openid获取到openid:', openid);
440:     } else if (loginRes.result && loginRes.result.uid) {
441:       openid = loginRes.result.uid;
442:       console.log('☐ [调试] 从result.uid获取到openid:', openid);
443:     } else {
444:       console.error('☐ [登录流程] 无法从login云函数返回数据中获取openid');
445:       console.error('☐ [登录流程] 完整返回数据:', JSON.stringify(loginRes, null, 2));
446:       throw new Error('云函数 login 未返回 openid');
447:     }
448:     console.log('☐ [云函数 login] 调用成功, openid: ', openid);
449:     this.globalData.openid = openid;
450:     uni.setStorageSync('userOpenId', openid); // 缓存 openid
451:
452:     // 步骤三：根据 openid 查询用户数据库
453:     // 【修正】使用 this.$tcb.database() 获取数据库实例
454:     const db = this.$tcb.database();
455:     const userRes = await db.collection('users').where({
456:       _openid: openid
457:     }).get();
458:
459:     if (userRes.data.length > 0) {
460:       // 用户已存在，登录成功
461:       const userInfo = userRes.data[0];
462:       console.log('☐ [数据库查询] 用户已注册，登录成功: ', userInfo);
463:       this.globalData.userInfo = userInfo;
464:       uni.setStorageSync('userInfo', userInfo); // 写入缓存
465:     } else {
466:       // 用户不存在，是新用户
467:       console.log('☐ [数据库查询] 新用户，尚未注册');
468:       this.globalData.userInfo = null; // 确保 userInfo 为 null
469:     }
470:
471:     // 无论新旧用户，都更新 getApp() 的 globalData
472:     const appInstance = getApp();

```

```

473:         if (appInstance) {
474:             appInstance.globalData = appInstance.globalData || {};
475:             appInstance.globalData.userInfo = this.globalData.userInfo;
476:             appInstance.globalData.openid = this.globalData.openid;
477:             appInstance.globalData._loginProcessCompleted = true; // 标记登录流程已完成
478:             console.log('☐ [登录流程] getApp().globalData 已更新:', appInstance.globalData);
479:         } else {
480:         }
481:
482:     } catch (err) {
483:         // 处理登录错误
484:
485:         // 即使登录失败，也标记登录流程已完成，避免后续显示登录提示
486:         const appInstance = getApp();
487:         if (appInstance) {
488:             appInstance.globalData = appInstance.globalData || {};
489:             appInstance.globalData._loginProcessCompleted = true;
490:         }
491:
492:         uni.showToast({
493:             icon: 'none',
494:             title: '登录失败，请稍后重试'
495:         });
496:     }
497: },
498:
499:     }
500: };
501:
502: </script>
503:
504: <style>
505: /* 全局字体预加载 - 确保Huiwen-mincho字体在所有页面都能立即显示 */
506: @font-face {
507:     font-family: 'Huiwen-mincho';
508:     src: url('/static/fonts/Huiwen-mincho.otf') format('opentype');
509:     font-weight: normal;
510:     font-style: normal;
511:     font-display: swap; /* 优化字体加载性能 */
512: }
513:
514: /* 全局样式：修改下拉刷新的loading转圈圈颜色为黑色 */
515: /* 针对微信小程序 */
516: /* #ifdef MP-WEIXIN */
517: .wx-pull-refresh {
518:     color: #000000 !important;
519: }
520:
521: .wx-pull-refresh .wx-pull-refresh-spinner {
522:     color: #000000 !important;

```

```
523: }
524:
525: .wx-pull-refresh .wx-pull-refresh-spinner::before {
526:     color: #000000 !important;
527: }
528: /* #endif */
529:
530: /* 针对H5 */
531: /* #ifdef H5 */
532: .uni-pull-refresh {
533:     color: #000000 !important;
534: }
535:
536: .uni-pull-refresh .uni-pull-refresh-spinner {
537:     color: #000000 !important;
538: }
539:
540: .uni-pull-refresh .uni-pull-refresh-spinner::before {
541:     color: #000000 !important;
542: }
543: /* #endif */
544:
545: /* 针对App */
546: /* #ifdef APP-PLUS */
547: .uni-pull-refresh {
548:     color: #000000 !important;
549: }
550:
551: .uni-pull-refresh .uni-pull-refresh-spinner {
552:     color: #000000 !important;
553: }
554:
555: .uni-pull-refresh .uni-pull-refresh-spinner::before {
556:     color: #000000 !important;
557: }
558: /* #endif */
559:
560: /* 通用样式: 修改所有下拉刷新的loading颜色 */
561: .uni-pull-refresh,
562: .wx-pull-refresh,
563: .pull-refresh {
564:     color: #000000 !important;
565: }
566:
567: .uni-pull-refresh .uni-pull-refresh-spinner,
568: .wx-pull-refresh .wx-pull-refresh-spinner,
569: .pull-refresh .pull-refresh-spinner {
570:     color: #000000 !important;
571:     border-color: #000000 !important;
572: }
```

```
573:
574: .uni-pull-refresh .uni-pull-refresh-spinner::before,
575: .wx-pull-refresh .wx-pull-refresh-spinner::before,
576: .pull-refresh .pull-refresh-spinner::before {
577:     color: #000000 !important;
578:     border-color: #000000 !important;
579: }
580:
581: /* 修改下拉刷新指示器的颜色 */
582: .uni-pull-refresh-indicator,
583: .wx-pull-refresh-indicator {
584:     color: #000000 !important;
585: }
586:
587: .uni-pull-refresh-indicator .uni-pull-refresh-spinner,
588: .wx-pull-refresh-indicator .wx-pull-refresh-spinner {
589:     color: #000000 !important;
590:     border-color: #000000 !important;
591: }
592: </style>
```

===== FILE: _utils/cache-manager.js =====

```
1: /**
2:  * 兼容层：重定向到新的缓存模块位置
3:  * @deprecated 请使用 @/cache 代替
4:  */
5: module.exports = require('../cache/core/manager');
```

===== FILE: _utils/compress.js =====

```
1: // compressImage.js
2: /**
3:  * @param {object} img 包含path: 图片的path, size: 图片的大小
4:  * @param {object} canvas canvas对象
5:  * @param {number} fileLimit 文件大小限制
6:  * @returns {Promise} 返回Promise对象
7:  */
8: function _compressImage(img, canvas, fileLimit) {
9:     return uni.getSystemInfo().then((res) => {
10:         let {
11:             // 设备像素比
12:             pixelRatio,
13:             // 设备品牌
14:             system
15:         } = res;
16:         // 是否是IOS系统
17:         let isIOS = /(ios)/gi.test(system);
18:         // 文件限制
19:         fileLimit = fileLimit || 2097152;
20:         // 基础大小 - 降低到1024以减小文件大小
21:         let baseSize = 1024;
```

```

22:         // 大于文件限制, 手动压缩
23:         if (img.size > fileLimit) {
24:             return compressImg({
25:                 src: img.path,
26:                 size: img.size,
27:                 canvas,
28:                 baseSize,
29:                 isIOS,
30:                 pixelRatio
31:             }).then((response) => {
32:                 return Promise.resolve(response);
33:             });
34:         }
35:         return Promise.resolve(img.path);
36:     });
37: }
38:
39: /**
40:  * @description 根据图片的大小选择压缩的方式
41:  * @param {string} src 图片的path
42:  * @param {number} size 图片的大小
43:  * @param {object} canvas canvas对象
44:  * @param {number} baseSize 基础尺寸
45:  * @param {boolean} isIOS 是否是IOS系统
46:  * @returns {Promise} 返回Promise对象
47:  */
48: function compressImg({ src, size, canvas, baseSize, isIOS, pixelRatio }) {
49:     return new Promise((resolve, reject) => {
50:         uni.getImageInfo({
51:             src
52:         })
53:         .then((res) => {
54:             let imgWidth = res.width;
55:             let imgHeight = res.height;
56:             if (imgWidth <= 4096 && imgHeight <= 4096) {
57:                 // 小于4096使用canvas压缩
58:                 canvasToImage({
59:                     src,
60:                     size,
61:                     imgWidth,
62:                     imgHeight,
63:                     canvas,
64:                     baseSize,
65:                     isIOS,
66:                     pixelRatio
67:                 }).then((response) => {
68:                     resolve(response);
69:                 });
70:             } else {
71:                 // 超过4096使用强制压缩

```

```

72:         compressImage(src, size, isIOS).then((response) => {
73:             resolve(response);
74:         });
75:     }
76: })
77: .catch((err) => {
78:     // 使用强制压缩
79:     compressImage(src, size, isIOS).then((response) => {
80:         resolve(response);
81:     });
82: });
83: });
84: }
85:
86: /**
87:  * @description 使用wx.compressImage压缩图片
88:  * @param {string} src 图片的path
89:  * @param {number} size 图片的大小
90:  * @param {boolean} isIOS 是否是IOS系统
91:  * @returns {Promise} 返回Promise对象
92:  */
93: function compressImage(src, size, isIOS) {
94:     return new Promise((resolve, reject) => {
95:         let quality = 100;
96:         if (isIOS) {
97:             quality = 0.08; // 进一步降低iOS压缩质量
98:         } else {
99:             // 更激进的压缩算法
100:             let temp = 25 - size / 1024 / 1024;
101:             quality = temp < 8 ? 8 : temp; // 最低质量从10降低到8
102:         }
103:
104:         uni.compressImage({
105:             src,
106:             quality,
107:             success: (res) => {
108:                 resolve(res.tempFilePath);
109:             },
110:             fail: () => {
111:                 // 压缩失败返回传递的图片src
112:                 resolve(src);
113:             }
114:         });
115:     });
116: }
117:
118: /**
119:  * @description 使用canvans压缩图片
120:  * @param {string} src 图片的path
121:  * @param {number} size 图片的大小

```



```

122: * @param {number} imgWidth 图片的宽度
123: * @param {number} imgHeight 图片的高度
124: * @param {object} canvas canvas对象
125: * @param {number} baseSize 基础尺寸
126: * @param {boolean} isIOS 是否是IOS系统
127: * @param {number} pixelRatio 设备像素比
128: * @returns {Promise} 返回Promise对象
129: */
130: function canvasToImage({ src, size, imgWidth, imgHeight, canvas, baseSize, isIOS, pixelRatio }) {
131:     return new Promise((resolve, reject) => {
132:         if (!canvas) {
133:             compressImage(src, size).then((res) => {
134:                 resolve(res);
135:             });
136:             return;
137:         }
138:         // 设置canvas宽度和高度
139:         let canvasWidth = 0;
140:         let canvasHeight = 0;
141:         let quality = 1;
142:         // 图片的宽度和高度都小于baseSize, 宽高不变
143:         if (imgWidth <= baseSize && imgHeight <= baseSize) {
144:             canvasWidth = imgWidth;
145:             canvasHeight = imgHeight;
146:             quality = 0.25; // 进一步降低质量
147:         } else {
148:             let compareFlag = true;
149:             // 图片的一边大于baseSize, 宽高不变
150:             if (pixelRatio > 2 && (imgWidth > baseSize || imgHeight > baseSize) && (imgWidth < baseSize || imgHeight < ba...
151:                 canvasWidth = imgWidth;
152:                 canvasHeight = imgHeight;
153:                 quality = 0.25; // 进一步降低质量
154:             } else {
155:                 // 按照原图的宽高比压缩
156:                 compareFlag = pixelRatio > 2 ? imgWidth > imgHeight : imgWidth > imgHeight;
157:                 // 宽比高大, 宽按基准比例缩放, 高设置为基准值, 高比宽大, 高按基准比例缩放, 宽设置为基准值。
158:                 canvasWidth = compareFlag ? parseInt(imgWidth / (imgHeight / baseSize)) : baseSize;
159:                 canvasHeight = compareFlag ? baseSize : parseInt(imgHeight / (imgwidth / baseSize));
160:                 quality = 0.7; // 进一步降低质量
161:             }
162:         }
163:
164:         let pic = canvas.createImage();
165:         pic.src = src;
166:         pic.onerror = function () {
167:             // 加载失败使用强制压缩
168:             compressImage(src, size, isIOS).then((response) => {
169:                 resolve(response);
170:             });
171:         };

```

```

172:     pic.onload = function () {
173:         // 获取绘画上下文
174:         let ctx = canvas.getContext('2d');
175:         ctx.clearRect(0, 0, canvasWidth, canvasHeight);
176:         ctx.drawImage(pic, 0, 0, canvasWidth, canvasHeight);
177:         // 导出图片
178:         uni.canvasToTempFilePath({
179:             canvas,
180:             width: canvasWidth,
181:             height: canvasHeight,
182:             destHeight: canvasHeight,
183:             destWidth: canvasWidth,
184:             fileType: 'jpg',
185:             quality,
186:             success: (res) => {
187:                 resolve(res.tempFilePath);
188:             },
189:             fail: (err) => {
190:                 // 压缩失败使用强制压缩
191:                 compressImage(src, size, isIOS).then((response) => {
192:                     resolve(response);
193:                 });
194:             }
195:         });
196:     };
197: });
198: }
199:
200: /**
201:  * @description 循环压缩图片
202:  * @param {object} img 包含path: 图片的path, size: 图片的大小
203:  * @param {object} canvas canvas对象
204:  * @param {number} fileLimit 文件大小限制
205:  * @returns {Promise} 返回Promise对象
206:  */
207:
208: module.exports = {
209:     compressImage: _compressImage // 我们只导出这一个核心函数, 并给它起个好记的名字
210: };

```

===== FILE: _utils/file-url-cache.js =====

```

1: /**
2:  * 兼容层: 重定向到新的缓存模块位置
3:  * @deprecated 请使用 @/cache 代替
4:  */
5: import fileUrlCache from '../cache/core/file-url';
6: export default fileUrlCache;
7:

```

===== FILE: _utils/hydrate-temp-urls.js =====

```
1: /**
2:  * 兼容层: 重定向到新的缓存模块位置
3:  * @deprecated 请使用 @/cache 代替
4:  */
5: export { hydrateTempUrls, warmTempUrlsFromPosts } from '../cache/core/hydrate';
6:
```

===== FILE: api-cache/cache-key-builder.js =====

```
1: /**
2:  * 兼容层: 重定向到新的缓存模块位置
3:  * @deprecated 请使用 @/cache 代替
4:  */
5: export { buildCacheKey, parseCacheKey, isQueryMatch } from '@/cache/core/key-builder';
6:
7:
```

===== FILE: api-cache/collage.js =====

```
1: import { cloudCall } from '@/utils/cloudCall';
2:
3: /**
4:  * 获取拼贴诗词库
5:  * @param {Object} options
6:  * @param {number} options.limit 每组返回数量, 默认10, 范围3-20
7:  * @param {Array<string>} options.groups 词组类型, 默认 ['nouns', 'verbs', 'imagery']
8:  * @param {number} options.seed 可选, 前后端复现同一批次
9:  */
10: export function getCollageWords(options = {}) {
11:   const payload = Object.assign(
12:     {
13:       mode: 'words',
14:       limit: 10,
15:       groups: ['nouns', 'verbs', 'imagery']
16:     },
17:     options
18:   );
19:
20:   return cloudCall('getCollagePoetry', payload, {
21:     pageTag: 'collage',
22:     injectOpenId: false
23:   });
24: }
25:
26: // 备用: 获取拼贴诗列表
27: export function getCollageList({ page = 0, pageSize = 10 } = {}) {
28:   return cloudCall(
29:     'getCollagePoetry',
30:     { page, pageSize },
31:     { pageTag: 'collage' }
32:   );
33: }
```

===== FILE: api-cache/comment.js =====

```
1: /**
2:  * 评论相关API缓存层
3:  */
4: const { cloudCall } = require('../utils/cloudCall.js');
5:
6: /**
7:  * 获取评论列表
8:  * @param {string} postId - 帖子ID
9:  * @param {Object} options - 额外选项
10:  * @returns {Promise} 评论列表数据
11:  */
12: function getComments(postId, options = {}) {
13:   if (!postId) {
14:     return Promise.reject(new Error('帖子ID不能为空'));
15:   }
16:
17:   return cloudCall('getComments', {
18:     postId: postId
19:   }, Object.assign({
20:     injectOpenId: true,
21:     pageTag: 'post-detail',
22:     ...options
23:   }));
24: }
25:
26: /**
27:  * 提交评论
28:  * @param {Object} commentData - 评论数据
29:  * @param {string} commentData.postId - 帖子ID
30:  * @param {string} commentData.content - 评论内容
31:  * @param {Array} commentData.images - 评论图片, 格式为 [{url, originalUrl, order}]
32:  * @param {string} commentData.parentId - 父评论ID (回复时使用)
33:  * @param {string} commentData.replyToAuthorName - 被回复者名称
34:  * @param {boolean} commentData.isAnonymous - 是否匿名评论
35:  * @param {Object} options - 额外选项
36:  * @returns {Promise} 提交结果
37:  */
38: function submitComment(commentData, options = {}) {
39:   if (!commentData) {
40:     return Promise.reject(new Error('评论数据不能为空'));
41:   }
42:
43:   const { postId, content, images, parentId, replyToAuthorName, isAnonymous } = commentData;
44:
45:   if (!postId) {
46:     return Promise.reject(new Error('帖子ID不能为空'));
47:   }
48: }
```

```

49:     const hasContent = content && content.trim().length > 0;
50:     const hasImages = images && images.length > 0;
51:
52:     if (!hasContent && !hasImages) {
53:         return Promise.reject(new Error('评论内容不能为空'));
54:     }
55:
56:     // 将 images 数组对象格式转换为云函数期望的 imageUrl 和 originalImageUrls 数组
57:     let imageUrl = [];
58:     let originalImageUrls = [];
59:     if (hasImages) {
60:         imageUrl = images.map(img => img.url || img).filter(Boolean);
61:         originalImageUrls = images.map(img => img.originalUrl || img.url || img).filter(Boolean);
62:     }
63:
64:     return cloudCall('addComment', {
65:         postId: postId,
66:         content: content ? content.trim() : '',
67:         imageUrl: imageUrl,
68:         originalImageUrls: originalImageUrls,
69:         parentId: parentId || null,
70:         replyToAuthorName: replyToAuthorName || null,
71:         isAnonymous: isAnonymous || false
72:     }, Object.assign({
73:         pageTag: 'post-detail',
74:         requireAuth: true,
75:         ...options
76:     })));
77: }
78:
79: /**
80:  * 删除评论
81:  * @param {string} commentId - 评论ID
82:  * @param {string} postId - 帖子ID
83:  * @param {string} parentId - 父评论ID（删除回复时使用）
84:  * @param {Object} options - 额外选项
85:  * @returns {Promise} 删除结果
86:  */
87: function deleteComment(commentId, postId, parentId = null, options = {}) {
88:     if (!commentId) {
89:         return Promise.reject(new Error('评论ID不能为空'));
90:     }
91:
92:     if (!postId) {
93:         return Promise.reject(new Error('帖子ID不能为空'));
94:     }
95:
96:     return cloudCall('deleteComment', {
97:         commentId: commentId,
98:         postId: postId,

```

```

99:         parentId: parentId
100:     }, Object.assign({
101:         pageTag: 'post-detail',
102:         requireAuth: true,
103:         ...options
104:     }));
105: }
106:
107: /**
108:  * 点赞评论
109:  * @param {string} commentId - 评论ID
110:  * @param {string} postId - 帖子ID
111:  * @param {boolean} isLiked - 是否点赞（可选，云函数会自动切换）
112:  * @param {Object} options - 额外选项
113:  * @returns {Promise} 点赞结果
114:  */
115: function likeComment(commentId, postId, isLiked, options = {}) {
116:     if (!commentId) {
117:         return Promise.reject(new Error('评论ID不能为空'));
118:     }
119:     if (!postId) {
120:         return Promise.reject(new Error('帖子ID不能为空'));
121:     }
122:
123:     return cloudCall('likeComment', {
124:         commentId: commentId,
125:         postId: postId,
126:         isLiked: isLiked
127:     }, Object.assign({
128:         pageTag: 'post-detail',
129:         requireAuth: true,
130:         ...options
131:     }));
132: }
133:
134: /**
135:  * 获取评论详情
136:  * @param {string} commentId - 评论ID
137:  * @param {Object} options - 额外选项
138:  * @returns {Promise} 评论详情
139:  */
140: function getCommentDetail(commentId, options = {}) {
141:     if (!commentId) {
142:         return Promise.reject(new Error('评论ID不能为空'));
143:     }
144:
145:     return cloudCall('getCommentDetail', {
146:         commentId: commentId
147:     }, Object.assign({
148:         injectOpenId: true,

```

```

149:         pageTag: 'post-detail',
150:         ...options
151:     }));
152: }
153:
154: /**
155:  * 举报评论
156:  * @param {string} commentId - 评论ID
157:  * @param {string} reason - 举报原因
158:  * @param {Object} options - 额外选项
159:  * @returns {Promise} 举报结果
160:  */
161: function reportComment(commentId, reason, options = {}) {
162:     if (!commentId) {
163:         return Promise.reject(new Error('评论ID不能为空'));
164:     }
165:
166:     if (!reason || reason.trim().length === 0) {
167:         return Promise.reject(new Error('举报原因不能为空'));
168:     }
169:
170:     return cloudCall('reportComment', {
171:         commentId: commentId,
172:         reason: reason.trim()
173:     }, Object.assign({
174:         pageTag: 'post-detail',
175:         requireAuth: true,
176:         ...options
177:     }));
178: }
179:
180: /**
181:  * 获取评论回复列表
182:  * @param {string} commentId - 评论ID
183:  * @param {Object} options - 额外选项
184:  * @returns {Promise} 回复列表
185:  */
186: function getCommentReplies(commentId, options = {}) {
187:     if (!commentId) {
188:         return Promise.reject(new Error('评论ID不能为空'));
189:     }
190:
191:     return cloudCall('getCommentReplies', {
192:         commentId: commentId
193:     }, Object.assign({
194:         injectOpenId: true,
195:         pageTag: 'post-detail',
196:         ...options
197:     }));
198: }

```

```

199:
200: module.exports = {
201:   getComments,
202:   submitComment,
203:   deleteComment,
204:   likeComment,
205:   getCommentDetail,
206:   reportComment,
207:   getCommentReplies
208: };

```

===== FILE: api-cache/discover.js =====

```

1: import cacheManager from '@/_utils/cache-manager';
2: const { cloudCall } = require('@/utils/cloudCall.js');
3:
4: // 发现页推荐: TTL 90s + SWR 45s (范围内可调整)
5: const TTL_MS = 90 * 1000;
6: const SWR_MS = 45 * 1000;
7:
8: const ns = cacheManager.namespace('posts:discover', { persistent: true, maxItems: 64 });
9:
10: function makeExcludeKey(arr) {
11:   if (!Array.isArray(arr) || arr.length === 0) return 'ex:none';
12:   try {
13:     const uniq = Array.from(new Set(arr.map(String)));
14:     uniq.sort();
15:     // 为避免 key 过长, 只保留前 50 个 (足够覆盖单页去重场景)
16:     const head = uniq.slice(0, 50);
17:     return 'ex:' + head.join(',');
18:   } catch (_) {
19:     return 'ex:err';
20:   }
21: }
22:
23: export async function getDiscoverFeed({ excludePostIds = [], page = 0, pageSize = 5, context, onBackgroundUpdate } = {}) {
24:   const key = `page:${page}:${makeExcludeKey(excludePostIds)}`;
25:   return ns.getOrFetch(
26:     key,
27:     async () => {
28:       const res = await cloudCall('getRecommendationFeed', {
29:         limit: pageSize,
30:         personalizedLimit: 3,
31:         hotLimit: 2,
32:         skip: 0,
33:         excludePostIds: excludePostIds || []
34:       }, { pageTag: 'discover', context });
35:       if (res && res.result && res.result.success) {
36:         return {
37:           posts: res.result.posts || [],
38:           hasMore: !!res.result.hasMore

```



```

39:     };
40:   }
41:   return { posts: [], hasMore: false };
42: },
43: {
44:   ttlMs: TTL_MS,
45:   swrMs: SWR_MS,
46:   onBackgroundUpdate
47: }
48: );
49: }
50:
51: export function invalidateDiscover() {
52:   ns.clear();
53: }
54:
55: export function clearDiscoverCache() {
56:   ns.clear();
57: }
58:

```

===== FILE: api-cache/discussion.js =====

```

1: import cacheManager from '@/_utils/cache-manager';
2: import { buildCacheKey } from './cache-key-builder.js';
3: const { cloudCall } = require('@/utils/cloudCall.js');
4:
5: // 讨论页帖子: TTL 120s + SWR 60s (讨论内容更新相对较慢, 缓存时间可以较长)
6: const TTL_MS = 120 * 1000;
7: const SWR_MS = 60 * 1000;
8:
9: // 使用独立的 discussion:posts 命名空间
10: const ns = cacheManager.namespace('discussion:posts', { persistent: true, maxItems: 128 });
11:
12: /**
13:  * 获取讨论页的帖子列表
14:  * @param {Object} options
15:  * @param {number} options.page - 页码, 从0开始
16:  * @param {number} options.pageSize - 每页数量
17:  * @param {Object} options.context - 页面上下文
18:  * @param {boolean} options.forceRefresh - 是否强制刷新 (跳过缓存)
19:  * @param {Function} options.onBackgroundUpdate - SWR后台更新完成回调
20:  */
21: export async function getDiscussionPosts({
22:   page = 0,
23:   pageSize = 10,
24:   context,
25:   forceRefresh = false,
26:   onBackgroundUpdate
27: } = {}) {
28:   // 构建缓存键

```

```

29: const key = buildCacheKey({ page, pageSize });
30:
31: // 如果强制刷新, 使用时间戳作为缓存键的一部分来绕过缓存
32: const cacheKey = forceRefresh && page === 0 ? `${key}:ts:${Date.now()}` : key;
33:
34: console.log('☐ [discussion] 请求数据 - key:', cacheKey, 'forceRefresh:', forceRefresh);
35:
36: // 第一页且强制刷新时, 跳过缓存直接调用云函数
37: if (page === 0 && forceRefresh) {
38:   console.log('☐ [discussion] 第一页强制刷新, 跳过缓存直接调用云函数');
39:   const res = await cloudCall(
40:     'getDiscussionPosts',
41:     {
42:       skip: page * pageSize,
43:       limit: pageSize
44:     },
45:     { pageTag: 'discussion', context, requireAuth: false }
46:   );
47:   console.log('☐ [discussion] 云函数返回 - success:', res?.result?.success, 'posts数量:', res?.result?.posts?.length);
48:   if (res && res.result && res.result.success) {
49:     return res.result.posts || [];
50:   }
51:   return [];
52: }
53:
54: // 使用缓存
55: return ns.getOrFetch(
56:   cacheKey,
57:   async () => {
58:     console.log('☐ [discussion] 缓存未命中, 调用云函数 - key:', key);
59:     const res = await cloudCall(
60:       'getDiscussionPosts',
61:       {
62:         skip: page * pageSize,
63:         limit: pageSize
64:       },
65:       { pageTag: 'discussion', context, requireAuth: false }
66:     );
67:     console.log('☐ [discussion] 云函数返回 - success:', res?.result?.success, 'posts数量:', res?.result?.posts?.length...
68:     if (res && res.result && res.result.success) {
69:       return res.result.posts || [];
70:     }
71:     return [];
72:   },
73:   {
74:     ttlMs: TTL_MS,
75:     swrMs: SWR_MS,
76:     onBackgroundUpdate
77:   }
78: );

```

```

79: }
80:
81: /**
82:  * 清除讨论页帖子列表缓存
83:  * @param {Object} options
84:  * @param {number} options.page - 页码, 如果指定则只清除该页的缓存
85:  * @param {number} options.pageSize - 每页数量
86:  */
87: export function invalidateDiscussionPosts({ page, pageSize = 10 } = {}) {
88:   if (typeof page === 'number') {
89:     // 清除特定页的缓存
90:     const key = buildCacheKey({ page, pageSize });
91:     ns.delete(key);
92:     console.log(`[discussion] 清除缓存 - key: ${key}`);
93:   } else {
94:     // 清除所有缓存
95:     ns.clear();
96:     console.log(`[discussion] 清除所有缓存`);
97:   }
98: }

```

===== FILE: api-cache/draft.js =====

```

1: /**
2:  * 草稿相关API缓存层
3:  */
4: const { cloudCall } = require('../utils/cloudCall.js');
5:
6: /**
7:  * 获取我的草稿列表
8:  * @param {Object} params - 查询选项
9:  * @returns {Promise} 草稿列表
10:  */
11: async function getMyDrafts(params = {}) {
12:   // 将解构移入函数体内, 解决 ReferenceError 问题
13:   const {
14:     page = 0,
15:     pageSize = 20,
16:     context,
17:     ...options
18:   } = params;
19:
20:   return cloudCall('getMyProfileData', {
21:     action: 'getDrafts',
22:     page: page,
23:     pageSize: pageSize
24:   }, Object.assign({
25:     pageTag: 'draft-box',
26:     requireAuth: true
27:   }, options));
28: }

```

```
29:
30: /**
31:  * 保存草稿
32:  * @param {Object} draftData - 草稿数据
33:  * @param {Object} options - 额外选项
34:  * @returns {Promise} 保存结果
35:  */
36: function saveDraft(draftData, options = {}) {
37:   if (!draftData) {
38:     return Promise.reject(new Error('草稿数据不能为空'));
39:   }
40:
41:   return cloudCall('getMyProfileData', {
42:     action: 'saveDraft',
43:     draftData: draftData
44:   }, Object.assign({
45:     pageTag: 'draft-box',
46:     requireAuth: true,
47:     ...options
48:   }));
49: }
50:
51: /**
52:  * 删除草稿
53:  * @param {string} draftId - 草稿ID
54:  * @param {Object} options - 额外选项
55:  * @returns {Promise} 删除结果
56:  */
57: function deleteDraft(draftId, options = {}) {
58:   if (!draftId) {
59:     return Promise.reject(new Error('草稿ID不能为空'));
60:   }
61:
62:   return cloudCall('getMyProfileData', {
63:     action: 'deleteDraft',
64:     draftId: draftId
65:   }, Object.assign({
66:     pageTag: 'draft-box',
67:     requireAuth: true,
68:     ...options
69:   }));
70: }
71:
72: /**
73:  * 清空所有草稿
74:  * @param {Object} options - 额外选项
75:  * @returns {Promise} 清空结果
76:  */
77: function clearAllDrafts(options = {}) {
78:   return cloudCall('getMyProfileData', {
```

```

79:   action: 'clearAllDrafts'
80: }, Object.assign({
81:   pageTag: 'draft-box',
82:   requireAuth: true,
83:   ...options
84: }));
85: }
86:
87: module.exports = {
88:   getMyDrafts,
89:   saveDraft,
90:   deleteDraft,
91:   clearAllDrafts
92: };

```

===== FILE: api-cache/events.js =====

```

1: import { EVENTS } from '@utils/events.js';
2: import { invalidateHomePosts } from '@api-cache/home-posts.js';
3: import { invalidatePostList } from '@api-cache/post-list.js';
4: import { clearDiscoverCache } from '@api-cache/discover.js';
5: import { invalidateUserInfo, invalidateUserPosts } from '@api-cache/user-profile.js';
6:
7: export function setupCacheEventBridges() {
8:   try {
9:     const g = (typeof globalThis !== 'undefined') ? globalThis : window;
10:    if (g.__CACHE_EVENTS_BRIDGED__) return;
11:    if (!uni || typeof uni.$on !== 'function') return;
12:
13:    // 新帖发布：刷新首页与发现页相关缓存
14:    // 由于所有页面都使用 posts:list 命名空间，失效首页缓存会清除所有相关缓存
15:    uni.$on(EVENTS.POST_CREATED, (payload = {}) => {
16:      try {
17:        // 失效所有首页缓存（包括所有筛选条件）
18:        invalidateHomePosts({});
19:        // 失效所有帖子列表缓存（跨页面共享的缓存）
20:        invalidatePostList({});
21:      } catch ( ) {}
22:      try { clearDiscoverCache(); } catch ( ) {}
23:    });
24:
25:    // 头像更新：失效对应用户资料与TA的个人主页分页，同时失效公共列表缓存
26:    uni.$on(EVENTS.AVATAR_UPDATED, (payload = {}) => {
27:      const uid = payload && (payload.userId || payload.userID || payload.uid);
28:      if (!uid) return;
29:      try { invalidateUserInfo(uid); } catch ( ) {}
30:      try { invalidateUserPosts(uid); } catch ( ) {}
31:      // 失效公共列表缓存，确保下次查询列表时能看到更新后的头像昵称
32:      // 由于所有页面都使用 posts:list 命名空间，失效首页缓存会清除所有相关缓存
33:      try {
34:        invalidateHomePosts({});

```

```

35:         // 失效所有帖子列表缓存（跨页面共享的缓存）
36:         invalidatePostList({});
37:     } catch ( ) {}
38:     try { clearDiscoverCache(); } catch ( ) {}
39:     // 失效所有标签页缓存（标签页也使用 posts:list 命名空间，但为了兼容性保留此逻辑）
40:     try {
41:         const { invalidateTagPosts } = require('@api-cache/tag-posts.js');
42:         // 由于标签页也使用 posts:list 命名空间，上面的 invalidatePostList 已经失效了
43:         // 这里保留代码以防未来有独立的标签命名空间
44:     } catch ( ) {}
45: });
46:
47: // 收藏变更：当前阶段按用户要求暂不做整页失效
48: // uni.$on(EVENTS.FAVORITE_CHANGED, () => { /* no-op */ });
49:
50: // 点赞变更：更新 like:status 专用缓存并同步到列表缓存
51: uni.$on(EVENTS.LIKE_CHANGED, (payload = {}) => {
52:     try {
53:         const postId = payload.postId;
54:         const votes = payload.votes;
55:         const isLiked = payload.isLiked;
56:         if (!postId) return;
57:         // 使用统一的点赞状态缓存
58:         try {
59:             const likeStatusCache = require('@cache/stores/like-status');
60:             likeStatusCache.updateLikeStatus(postId, votes, isLiked);
61:             likeStatusCache.syncToListCaches([postId]);
62:         } catch ( ) {}
63:     } catch ( ) {}
64: });
65:
66: // 评论点赞变更：仅维护专用缓存，页面自行监听精确更新
67: uni.$on(EVENTS.COMMENT_LIKE_CHANGED, (payload = {}) => {
68:     try {
69:         const { commentId, likes, liked } = payload || {};
70:         if (!commentId) return;
71:         try { const { updateCommentLikeCache, persistToStorage } = require('@utils/commentLikeStatusSync.js');
72:             updateCommentLikeCache(commentId, likes, liked);
73:             persistToStorage(commentId, likes, liked);
74:         } catch ( ) {}
75:     } catch ( ) {}
76: });
77:
78: // 评论数变更：更新各列表缓存中对应 post 的 commentCount
79: uni.$on(EVENTS.COMMENT_COUNT_CHANGED, (payload = {}) => {
80:     try {
81:         const postId = payload && payload.postId;
82:         const commentCount = (payload && typeof payload.commentCount === 'number') ? payload.commentCount : undefined;
83:         if (!postId || typeof commentCount !== 'number') return;
84:         try {

```

```

121:         for (let i = 0; i < line.length; i++) {
122:             const char = line[i];
123:             const testLine = currentLine + char;
124:             const testWidth = ctx.measureText ? ctx.measureText(testLine).width : testLine.length * fontSize * 0.6;
125:             // 优化: 给Canvas文本测量增加5%的容错, 避免过于保守的换行
126:             if (testWidth > maxWidth * 0.95 && currentLine) {
127:                 wrappedLines.push(currentLine);
128:                 currentLine = char;
129:             } else {
130:                 currentLine = testLine;
131:             }
132:         }
133:         if (currentLine) wrappedLines.push(currentLine);
134:     }
135: });
136:
137:     return wrappedLines;
138: }
139:
140: /**
141:  * 防止单字换行的中文排版优化
142:  * @param {string} text - 原文本
143:  * @returns {string}
144:  */
145: function preventShortLineBreak(text) {
146:     if (!text) return '';
147:     // 在标点前添加零空格防止单字换行
148:     return text.replace(/([,。！？\、\;""''')】』》])/g, '\u200B$1');
149: }
150:
151: /**
152:  * 绘制圆角矩形
153:  * @param {Object} ctx - Canvas 上下文
154:  * @param {number} x - x 坐标
155:  * @param {number} y - y 坐标
156:  * @param {number} width - 宽度
157:  * @param {number} height - 高度
158:  * @param {number} radius - 圆角半径
159:  */
160: function drawRoundRect(ctx, x, y, width, height, radius) {
161:     ctx.beginPath();
162:     ctx.moveTo(x + radius, y);
163:     ctx.lineTo(x + width - radius, y);
164:     ctx.arcTo(x + width, y, x + width, y + radius, radius);
165:     ctx.lineTo(x + width, y + height - radius);
166:     ctx.arcTo(x + width, y + height, x + width - radius, y + height, radius);
167:     ctx.lineTo(x + radius, y + height);
168:     ctx.arcTo(x, y + height, x, y + height - radius, radius);
169:     ctx.lineTo(x, y + radius);
170:     ctx.arcTo(x, y, x + radius, y, radius);

```

```
171:     ctx.closePath();
172: }
173:
174: /**
175:  * 绘制多行文字
176:  * @param {Object} ctx - Canvas 上下文
177:  * @param {string[]} lines - 行数组
178:  * @param {number} x - x 坐标
179:  * @param {number} startY - 起始 y 坐标
180:  * @param {number} lineHeight - 行高
181:  * @param {string} color - 颜色
182:  * @returns {number} 结束 y 坐标
183:  */
184: function drawMultiLineText(ctx, lines, x, startY, lineHeight, color = '#333') {
185:     ctx.setFillStyle(color);
186:     let y = startY;
187:     lines.forEach(line => {
188:         ctx.fillText(line, x, y);
189:         y += lineHeight;
190:     });
191:     return y;
192: }
193:
194: /**
195:  * 加载字体
196:  * @param {string} family - 字体名
197:  * @param {string} source - 字体源
198:  * @returns {Promise<boolean>}
199:  */
200: function loadFont(family, source) {
201:     return new Promise((resolve) => {
202:         uni.loadFontFace({
203:             family,
204:             source,
205:             success: () => resolve(true),
206:             fail: () => resolve(false)
207:         });
208:     });
209: }
210:
211: /**
212:  * 导出 Canvas 为图片
213:  * @param {string} canvasId - Canvas ID
214:  * @param {Object} context - 组件上下文
215:  * @param {number} width - 宽度
216:  * @param {number} height - 高度
217:  * @returns {Promise<string>}
218:  */
219: function canvasToTempFile(canvasId, context, width, height) {
220:     return new Promise((resolve, reject) => {
```



```

221:         uni.canvasToTempFilePath({
222:             canvasId,
223:             x: 0,
224:             y: 0,
225:             width,
226:             height,
227:             destWidth: width * 2,
228:             destHeight: height * 2,
229:             fileType: 'jpg',
230:             quality: 0.9,
231:             success: (res) => resolve(res.tempFilePath),
232:             fail: (err) => reject(err)
233:         }, context);
234:     });
235: }
236:
237: /**
238:  * 绘制圆角矩形 (使用 quadraticCurveTo)
239:  */
240: function drawRoundedRect(ctx, x, y, width, height, radius) {
241:     ctx.beginPath();
242:     ctx.moveTo(x + radius, y);
243:     ctx.lineTo(x + width - radius, y);
244:     ctx.quadraticCurveTo(x + width, y, x + width, y + radius);
245:     ctx.lineTo(x + width, y + height - radius);
246:     ctx.quadraticCurveTo(x + width, y + height, x + width - radius, y + height);
247:     ctx.lineTo(x + radius, y + height);
248:     ctx.quadraticCurveTo(x, y + height, x, y + height - radius);
249:     ctx.lineTo(x, y + radius);
250:     ctx.quadraticCurveTo(x, y, x + radius, y);
251:     ctx.closePath();
252: }
253:
254: /**
255:  * 绘制右下角水印
256:  */
257: function drawCornerWatermark(ctx, canvasWidth, canvasHeight) {
258:     const margin = 24;
259:     const w = 220;
260:     const h = 180;
261:     const stepX = 0.55;
262:     const stepY = 0.60;
263:     const x0 = canvasWidth - w - margin;
264:     const y0 = canvasHeight - h - margin;
265:     const lineWidth = 1.5;
266:     const strokeColor = 'rgba(0,0,0,0.16)';
267:     const textColor = 'rgba(0,0,0,0.22)';
268:     const sx = x0 + w * stepX;
269:     const sy = y0 + h * stepY;
270:

```

```

271:     ctx.save();
272:     try {
273:         if (ctx.setLineWidth) ctx.setLineWidth(lineWidth); else ctx.lineWidth = lineWidth;
274:         if (ctx.setStrokeStyle) ctx.setStrokeStyle(strokeColor); else ctx.strokeStyle = strokeColor;
275:         // 顶边
276:         ctx.beginPath();
277:         ctx.moveTo(sx, y0);
278:         ctx.lineTo(x0 + w, y0);
279:         ctx.stroke();
280:         // 左边
281:         ctx.beginPath();
282:         ctx.moveTo(x0, sy);
283:         ctx.lineTo(x0, y0 + h);
284:         ctx.stroke();
285:         // 内横线
286:         ctx.beginPath();
287:         ctx.moveTo(x0, sy);
288:         ctx.lineTo(sx, sy);
289:         ctx.stroke();
290:         // 内竖线
291:         ctx.beginPath();
292:         ctx.moveTo(sx, y0);
293:         ctx.lineTo(sx, sy);
294:         ctx.stroke();
295:         // 小字"poementer"
296:         try {
297:             const inset = 12;
298:             if (ctx.setFillStyle) ctx.setFillStyle(textColor); else ctx.fillStyle = textColor;
299:             const fontPx = 18;
300:             try { ctx.font = fontPx + 'px 汇文明朝, sans-serif'; } catch (_) {}
301:             if (ctx.setFontSize) ctx.setFontSize(fontPx);
302:             if (ctx.setTextAlign) ctx.setTextAlign('left'); else ctx.textAlign = 'left';
303:             ctx.fillText('poementer', x0 + inset, y0 + h - inset);
304:         } catch (_) {}
305:     } finally { ctx.restore(); }
306: }
307:
308: /**
309:  * 计算分享卡片所需高度
310:  * @param {Object} options - 配置项
311:  * @returns {Promise<{canvasHeight: number, processedLines: string[], titleHeight: number, signatureDrawHeight: number}>}>
312:  */
313: async function calculateShareCardHeight(options) {
314:     const {
315:         measureCtx,
316:         post,
317:         shareConfig,
318:         canvasWidth = 750,
319:         shouldShowSignature = true
320:     } = options;

```

```

321:
322:   const baseFontSize = shareConfig.fontSize || 38;
323:   const fontScale = shareConfig.fontScale || 1.0;
324:   const fontSize = Math.round(baseFontSize * fontScale);
325:   const lineHeight = Math.round(fontSize * 1.26);
326:   const fontFamily = shareConfig.fontFamily || '汇文明朝';
327:   // 将字体 ID 转换为显示名称, 用于 Canvas 绑定
328:   const fontDisplayName = getFontDisplayName(fontFamily);
329:   const actualFontFamily = fontFamily === 'system' ? 'sans-serif' : fontDisplayName + ', sans-serif';
330:
331:   const textPadding = 60;
332:   const textTopPadding = 80;
333:   const textBottomPadding = 60;
334:
335:   const baseTitleFontSize = shareConfig.titleFontSize || Math.round(baseFontSize * 1.21);
336:   const titleFontSize = Math.round(baseTitleFontSize * fontScale);
337:   const titleLineHeight = Math.round(titleFontSize * 1.22);
338:   const titleBottomSpacing = Math.round(fontSize * 0.84);
339:
340:   const textAreaWidth = canvasWidth - 120;
341:   const content = post.content || '';
342:
343:   if (measureCtx) {
344:     measureCtx.font = fontSize + 'px ' + actualFontFamily;
345:   }
346:
347:   // 计算正文行数和高度
348:   const processedLines = wrapText(measureCtx, content, textAreaWidth, fontSize);
349:   const wrappedContentHeight = processedLines.reduce((h, line) => h + (line && line.trim() ? lineHeight : lineHeight * ...
350:   const contentHeight = Math.max(wrappedContentHeight, 200);
351:
352:   // 计算标题高度
353:   let actualTitleHeight = titleLineHeight + 20;
354:   if (post.title) {
355:     const titleLines = wrapText(measureCtx, post.title, textAreaWidth, titleFontSize);
356:     const titleLinesCount = titleLines.filter(line => line.trim()).length;
357:     if (titleLinesCount > 1) {
358:       actualTitleHeight = titleLinesCount * titleLineHeight + 20;
359:     }
360:   }
361:   const titleHeight = actualTitleHeight;
362:
363:   // 计算签名高度
364:   const signatureTopGap = 40;
365:   const fixedSignatureWidth = 120;
366:   const signatureTextFontSize = 28;
367:   let signatureDrawHeight = 0;
368:
369:   if (post.authorSignature && shouldShowSignature) {
370:     try {

```

```

371:         const sigInfo = await new Promise((resolve) => {
372:             uni.getImageInfo({ src: post.authorSignature, success: (res) => resolve(res), fail: () => resolve(null) }...
373:         });
374:         if (sigInfo && sigInfo.width > 0) {
375:             const scale = fixedSignatureWidth / sigInfo.width;
376:             signatureDrawHeight = Math.max(1, Math.round(sigInfo.height * scale));
377:         }
378:     } catch ( ) {}
379: }
380:
381: // 计算最终高度
382: const isNonOriginalPoem = post.isPoem && post.isOriginal === false && post.author;
383: const needsAuthorSpace = shouldShowSignature || isNonOriginalPoem;
384:
385: let finalCanvasHeight = textTopPadding + titleHeight + titleBottomSpacing + contentHeight
386:   + (post.authorSignature && shouldShowSignature
387:     ? (signatureTopGap + signatureDrawHeight)
388:     : (needsAuthorSpace && ((post.authorName && post.authorName.trim()) || (post.author && post.author.trim()))
389:       ? (signatureTopGap + signatureTextFontSize)
390:       : 0)
391:   + textBottomPadding + 10;
392:
393: const safetyMargin = Math.max(60, Math.ceil(lineHeight * 1.5));
394: finalCanvasHeight += safetyMargin;
395:
396: return {
397:     canvasHeight: Math.ceil(finalCanvasHeight),
398:     processedLines,
399:     titleHeight,
400:     signatureDrawHeight,
401:     fontSize,
402:     lineHeight,
403:     titleFontSize,
404:     titleLineHeight,
405:     titleBottomSpacing,
406:     actualFontFamily,
407:     textPadding,
408:     textTopPadding,
409:     textBottomPadding,
410:     textAreaWidth,
411:     signatureTopGap,
412:     fixedSignatureWidth,
413:     signatureTextFontSize
414: };
415: }
416:
417: /**
418:  * 绘制分享卡片内容
419:  * @param {Object} options - 配置项
420:  */

```

```
421: async function drawShareCardContent(options) {
422:   const {
423:     ctx,
424:     post,
425:     shareConfig,
426:     canvasWidth,
427:     canvasHeight,
428:     processedLines,
429:     titleHeight,
430:     signatureDrawHeight,
431:     fontSize,
432:     lineHeight,
433:     titleFontSize,
434:     titleLineHeight,
435:     titleBottomSpacing,
436:     actualFontFamily,
437:     textPadding,
438:     textTopPadding,
439:     textAreaWidth,
440:     signatureTopGap,
441:     fixedSignatureWidth,
442:     signatureTextFontSize,
443:     shouldShowSignature
444:   } = options;
445:
446:   // 设置字体
447:   ctx.font = fontSize + 'px ' + actualFontFamily;
448:
449:   // 【修复】先清空Canvas，确保没有残留
450:   ctx.clearRect(0, 0, canvasWidth, canvasHeight);
451:
452:   // 绘制圆角背景
453:   const bgColor = shareConfig.backgroundColor || post.backgroundColor || '#FFFFFF';
454:   ctx.setFillStyle(bgColor);
455:   drawRoundedRect(ctx, 0, 0, canvasWidth, canvasHeight, 15);
456:   ctx.fill();
457:
458:   // 绘制文字内容
459:   const textColor = shareConfig.textColor || post.textColor || '#000000';
460:   ctx.setFillStyle(textColor);
461:   ctx.setTextAlign('left');
462:
463:   // 绘制标题
464:   const title = post.title || '';
465:   if (title) {
466:     ctx.font = titleFontSize + 'px ' + actualFontFamily;
467:     ctx.setFillStyle(textColor);
468:     ctx.setTextAlign('left');
469:
470:     const titleLines = wrapText(ctx, title, textAreaWidth, titleFontSize, actualFontFamily);
```

```

471:     let titleY = textTopPadding + titleFontSize;
472:     const titleX = textPadding;
473:
474:     titleLines.forEach((line) => {
475:         if (line.trim()) {
476:             ctx.fillText(line, titleX, titleY);
477:             titleY += titleLineHeight;
478:         } else {
479:             titleY += titleLineHeight * 0.5;
480:         }
481:     });
482:
483:     ctx.font = fontSize + 'px ' + actualFontFamily;
484: }
485:
486: // 绘制正文
487: let y = textTopPadding + titleHeight + titleBottomSpacing + fontSize;
488: const x = textPadding;
489:
490: processedLines.forEach((line) => {
491:     if (line.trim()) {
492:         ctx.fillText(line, x, y);
493:         y += lineHeight;
494:     } else {
495:         y += lineHeight * 0.5;
496:     }
497: });
498:
499: // 绘制签名
500: if (post.authorSignature && shouldShowSignature) {
501:     const signatureMargin = 40;
502:     const signatureX = canvasWidth - fixedSignatureWidth - signatureMargin;
503:     const signatureY = Math.min(y + signatureTopGap, canvasHeight - signatureDrawHeight - signatureMargin);
504:
505:     await drawImageAsync(ctx, post.authorSignature, signatureX, signatureY, fixedSignatureWidth);
506: } else if (shouldShowSignature) {
507:     const authorName = ((post.authorName || post.author || '') + '').trim();
508:     if (authorName) {
509:         ctx.setTextAlign('right');
510:         ctx.setFillStyle(textColor);
511:         ctx.font = signatureTextFontSize + 'px ' + actualFontFamily;
512:         const textMargin = 48;
513:         const sigTextX = canvasWidth - textMargin;
514:         const sigTextY = Math.max(canvasHeight - textMargin, y + signatureTopGap);
515:         ctx.fillText(authorName, sigTextX, sigTextY);
516:         ctx.setTextAlign('left');
517:     }
518: } else if (post.isPoem && post.isOriginal === false && post.author) {
519:     const originalAuthor = (post.author + '').trim();
520:     if (originalAuthor) {

```

```

521:         ctx.setTextAlign('right');
522:         ctx.setFillStyle(textColor);
523:         ctx.font = signatureTextFontSize + 'px ' + actualFontFamily;
524:         const textMargin = 48;
525:         const sigTextX = canvasWidth - textMargin;
526:         const sigTextY = Math.max(canvasHeight - textMargin, y + signatureTopGap);
527:         ctx.fillText(originalAuthor, sigTextX, sigTextY);
528:         ctx.setTextAlign('left');
529:     }
530: }
531:
532: // 绘制水印
533: try { drawCornerWatermark(ctx, canvasWidth, canvasHeight); } catch (e) { console.warn('draw watermark failed', e); }
534: }
535:
536: module.exports = {
537:     drawImageAsync,
538:     calculateActualLines,
539:     wrapText,
540:     preventShortLineBreak,
541:     drawRoundRect,
542:     drawRoundedRect,
543:     drawMultiLineText,
544:     loadFont,
545:     canvasToTempFile,
546:     drawCornerWatermark,
547:     calculateShareCardHeight,
548:     drawShareCardContent
549: };

```

===== FILE: utils/shareImage.js =====

```

1: /**
2:  * 分享图片生成和处理工具函数
3:  */
4:
5: /**
6:  * 生成分享图片的文件名
7:  * @param {string} postId - 帖子ID
8:  * @param {string} userId - 用户ID
9:  * @returns {string} 分享图片文件名
10: */
11: function generateShareImageName(postId, userId) {
12:     const timestamp = Date.now();
13:     return `share_${postId}_${userId}_${timestamp}.jpg`;
14: }
15:
16: /**
17:  * 验证图片数据URL是否有效
18:  * @param {string} dataUrl - 图片数据URL
19:  * @returns {boolean} 是否有效

```

```

20: */
21: function isValidImageUrl(dataUrl) {
22:     if (!dataUrl || typeof dataUrl !== 'string') {
23:         return false;
24:     }
25:
26:     // 检查数据URL格式
27:     const dataUrlRegex = /^data:image\/(jpeg|jpg|png);base64,([A-Za-z0-9+\/=]+)$/;
28:     return dataUrlRegex.test(dataUrl);
29: }
30:
31: /**
32:  * 将base64数据转换为ArrayBuffer
33:  * @param {string} base64Data - base64数据
34:  * @returns {ArrayBuffer} ArrayBuffer数据
35:  */
36: function base64ToArrayBuffer(base64Data) {
37:     // 移除data:image/...;base64, 前缀
38:     const base64 = base64Data.replace(/^data:image\/[a-z]+;base64,/ , '');
39:
40:     // 解码base64
41:     const binaryString = atob(base64);
42:     const bytes = new Uint8Array(binaryString.length);
43:
44:     for (let i = 0; i < binaryString.length; i++) {
45:         bytes[i] = binaryString.charCodeAt(i);
46:     }
47:
48:     return bytes.buffer;
49: }
50:
51: /**
52:  * 保存图片到相册
53:  * @param {string} imagePath - 图片路径
54:  * @returns {Promise} 保存结果
55:  */
56: function saveImageToAlbum(imagePath) {
57:     return new Promise((resolve, reject) => {
58:         if (!imagePath) {
59:             reject(new Error('图片路径为空'));
60:             return;
61:         }
62:
63:         uni.saveImageToPhotosAlbum({
64:             filePath: imagePath,
65:             success: (res) => {
66:                 resolve(res);
67:             },
68:             fail: (err) => {
69:                 console.error('保存图片到相册失败:', err);

```



```

70:
71: // 如果用户拒绝授权，提示用户打开权限
72: if (err.errMsg.includes('auth deny') || err.errMsg.includes('auth denied')) {
73:   uni.showModal({
74:     title: '提示',
75:     content: '需要您授权保存图片到相册',
76:     showCancel: true,
77:     confirmText: '去授权',
78:     success: (modalRes) => {
79:       if (modalRes.confirm && uni.openSetting) {
80:         uni.openSetting({
81:           success: (settingRes) => {
82:             if (settingRes.authSetting['scope.writePhotosAlbum']) {
83:               // 用户已授权，重新保存
84:               saveImageToAlbum(imagePath)
85:                 .then(resolve)
86:                 .catch(reject);
87:             } else {
88:               reject(new Error('用户未授权保存图片到相册'));
89:             }
90:           }
91:         });
92:       } else {
93:         reject(new Error('用户未授权保存图片到相册'));
94:       }
95:     }
96:   });
97: } else {
98:   reject(err);
99: }
100: }
101: });
102: });
103: }
104:
105: /**
106:  * 创建临时文件路径
107:  * @param {string} fileName - 文件名
108:  * @returns {string} 临时文件路径
109:  */
110: function createTempFilePath(fileName) {
111:   const timestamp = Date.now();
112:   const randomSuffix = Math.random().toString(36).substring(2, 8);
113:   const tempFileName = fileName || `share_${timestamp}_${randomSuffix}.jpg`;
114:
115:   return `${uni.env.USER_DATA_PATH}/${tempFileName}`;
116: }
117:
118: /**
119:  * 压缩图片

```

```

120: * @param {string} imagePath - 原图片路径
121: * @param {number} quality - 压缩质量 (0-100)
122: * @param {number} maxWidth - 最大宽度
123: * @param {number} maxHeight - 最大高度
124: * @returns {Promise} 压缩后的图片路径
125: */
126: function compressImage(imagePath, quality = 80, maxWidth = 1080, maxHeight = 1920) {
127:   return new Promise((resolve, reject) => {
128:     if (!imagePath) {
129:       reject(new Error('图片路径为空'));
130:       return;
131:     }
132:
133:     uni.compressImage({
134:       src: imagePath,
135:       quality: quality,
136:       success: (res) => {
137:         resolve(res.tempFilePath);
138:       },
139:       fail: (err) => {
140:         console.error('压缩图片失败:', err);
141:         // 压缩失败则返回原图路径
142:         resolve(imagePath);
143:       }
144:     });
145:   });
146: }
147:
148: /**
149:  * 获取图片信息
150:  * @param {string} imagePath - 图片路径
151:  * @returns {Promise} 图片信息
152:  */
153: function getImageInfo(imagePath) {
154:   return new Promise((resolve, reject) => {
155:     if (!imagePath) {
156:       reject(new Error('图片路径为空'));
157:       return;
158:     }
159:
160:     uni.getImageInfo({
161:       src: imagePath,
162:       success: (res) => {
163:         resolve(res);
164:       },
165:       fail: (err) => {
166:         console.error('获取图片信息失败:', err);
167:         reject(err);
168:       }
169:     });

```

```

170:     });
171: }
172:
173: module.exports = {
174:     generateShareImageName,
175:     isValidImageUrl,
176:     base64ToArrayBuffer,
177:     saveImageToAlbum,
178:     createTempFilePath,
179:     compressImage,
180:     getImageInfo
181: };

```

===== FILE: utils/signatureCache.js =====

```

1: /**
2:  * 兼容层：重定向到新的缓存模块位置
3:  * @deprecated 请使用 @/cache 代替
4:  */
5: module.exports = require('../cache/stores/signature');

```

===== FILE: utils/tabBarCompatibility.js =====

```

1: /**
2:  * tabBar 兼容性处理工具
3:  * 解决 getTabBar() 在不同平台不支持的问题
4:  */
5:
6: /**
7:  * 安全地更新TabBar状态
8:  * @param {Object} pageInstance - 页面实例
9:  * @param {number} selectedIndex - 选中的tab索引
10:  */
11: export function updateTabBarStatus(pageInstance, selectedIndex) {
12:     try {
13:         // 只使用原生getTabBar API
14:         if (typeof pageInstance.getTabBar === 'function' && pageInstance.getTabBar()) {
15:             pageInstance.getTabBar().setData({
16:                 selected: selectedIndex
17:             });
18:             console.log(`TabBar状态更新成功 (原生API): selected=${selectedIndex}`);
19:             return true;
20:         }
21:     } catch (e) {
22:         console.log('原生getTabBar API不可用:', e.message);
23:     }
24:
25:     // 如果原生API不可用，使用存储机制作为备选
26:     try {
27:         uni.setStorageSync('currentTabIndex', selectedIndex);
28:         console.log(`TabBar状态已保存到存储: selected=${selectedIndex}`);
29:         return true;

```

```

30:     } catch (e) {
31:         console.log('存储机制不可用:', e.message);
32:     }
33:
34:     console.warn('原生getTabBar API不可用, 当前平台可能不支持getTabBar()');
35:     return false;
36: }
37:
38: /**
39:  * 获取当前TabBar状态
40:  * @param {Object} pageInstance - 页面实例
41:  * @returns {number|null} 当前选中的tab索引
42:  */
43: export function getCurrentTabBarStatus(pageInstance) {
44:     try {
45:         // 方法1: 从原生getTabBar获取
46:         if (typeof pageInstance.getTabBar === 'function' && pageInstance.getTabBar()) {
47:             const tabBar = pageInstance.getTabBar();
48:             return tabBar.data?.selected || null;
49:         }
50:     } catch (e) {
51:         console.log('无法从原生getTabBar获取状态');
52:     }
53:
54:     try {
55:         // 方法2: 从自定义组件获取
56:         const tabBar = pageInstance.$refs?.customTabBar;
57:         if (tabBar && tabBar.data) {
58:             return tabBar.data.selected || null;
59:         }
60:     } catch (e) {
61:         console.log('无法从自定义组件获取状态');
62:     }
63:
64:     try {
65:         // 方法3: 从存储获取
66:         return uni.getStorageSync('currentTabIndex') || null;
67:     } catch (e) {
68:         console.log('无法从存储获取状态');
69:     }
70:
71:     return null;
72: }
73:
74: /**
75:  * 检查当前平台是否支持getTabBar
76:  * @returns {boolean} 是否支持
77:  */
78: export function isTabBarSupported() {
79:     try {

```

```
80:         // 检查是否在小程序环境
81:         if (typeof wx !== 'undefined' && wx.getTabBar) {
82:             return true;
83:         }
84:
85:         // 检查是否有自定义tabBar
86:         if (typeof uni !== 'undefined' && uni.getTabBar) {
87:             return true;
88:         }
89:
90:         return false;
91:     } catch (e) {
92:         return false;
93:     }
94: }
```

===== FILE: utils/tagCategories.js =====

```
1: /**
2:  * 标签分类配置数据
3:  * 用于发布页面的标签选择器
4:  */
5:
6: export const tagCategories = [
7:     {
8:         name: '内容主题',
9:         icon: '□',
10:        tags: [
11:            '爱情',
12:            '亲情',
13:            '友情',
14:            '乡愁',
15:            '思念',
16:            '孤独',
17:            '咏物',
18:            '山水',
19:            '田园',
20:            '季节',
21:            '春天',
22:            '夏日',
23:            '秋风',
24:            '冬雪',
25:            '人生',
26:            '理想',
27:            '哲理',
28:            '时间',
29:            '青春',
30:            '成长',
31:            '生死',
32:            '怀古',
33:            '咏史',
```

```
34:         '边塞',
35:         '战争',
36:         '爱国',
37:         '城市',
38:         '乡村',
39:         '生活',
40:         '旅行',
41:         '饮食',
42:         '劳动'
43:     ]
44: },
45: {
46:     name: '情感基调',
47:     icon: '□',
48:     tags: [
49:         '治愈',
50:         '温暖',
51:         '浪漫',
52:         '唯美',
53:         '悲伤',
54:         '伤感',
55:         '惆怅',
56:         '寂寞',
57:         '豪放',
58:         '豁达',
59:         '激昂',
60:         '热血',
61:         '婉约',
62:         '细腻',
63:         '清新',
64:         '宁静',
65:         '励志',
66:         '鼓舞',
67:         '坚定',
68:         '充满希望',
69:         '讽刺',
70:         '批判',
71:         '深沉',
72:         '引人深思'
73:     ]
74: },
75: {
76:     name: '形式体裁',
77:     icon: '□',
78:     tags: [
79:         '古体诗',
80:         '近体诗',
81:         '五言',
82:         '七言',
83:         '绝句',
```

```
84:         '律诗',
85:         '词',
86:         '曲',
87:         '乐府',
88:         '骚体',
89:         '现代诗',
90:         '自由诗',
91:         '散文诗',
92:         '十四行诗',
93:         '叙事诗',
94:         '俳句',
95:         '短歌',
96:         '史诗',
97:         '长诗',
98:         '短诗',
99:         '微型诗',
100:        '三行诗'
101:    ]
102: },
103: {
104:     name: '意象元素',
105:     icon: '□',
106:     tags: [
107:         '月亮',
108:         '星星',
109:         '太阳',
110:         '宇宙',
111:         '银河',
112:         '风',
113:         '雨',
114:         '雪',
115:         '云',
116:         '雾',
117:         '河流',
118:         '大海',
119:         '山峰',
120:         '森林',
121:         '花',
122:         '草',
123:         '树',
124:         '麦田',
125:         '落叶',
126:         '梅',
127:         '兰',
128:         '竹',
129:         '菊',
130:         '鸟',
131:         '马',
132:         '蝉',
133:         '鱼',
```

```
134:         '蝴蝶',
135:         '酒',
136:         '剑',
137:         '琴',
138:         '灯',
139:         '船',
140:         '镜子',
141:         '红色',
142:         '白色',
143:         '蓝色',
144:         '金色'
145:     ]
146: },
147: {
148:     name: '风格流派',
149:     icon: '□',
150:     tags: [
151:         '唐诗',
152:         '宋词',
153:         '元曲',
154:         '先秦',
155:         '两汉',
156:         '魏晋',
157:         '建安风骨',
158:         '朦胧诗',
159:         '新月派',
160:         '浪漫主义',
161:         '现实主义',
162:         '象征主义',
163:         '现代主义',
164:         '意象派',
165:         '垮掉的一代',
166:         '中文诗',
167:         '英文诗',
168:         '日文诗',
169:         '法文诗',
170:         '翻译诗',
171:         '中国',
172:         '英国',
173:         '美国',
174:         '日本',
175:         '俄罗斯'
176:     ]
177: },
178: {
179:     name: '场景用途',
180:     icon: '□',
181:     tags: [
182:         '晚安诗',
183:         '早安问候',
```



```

184:         '节日祝福',
185:         '春节',
186:         '中秋',
187:         '情人节',
188:         '毕业季',
189:         '婚礼致辞',
190:         '旅行途中',
191:         '雨天读诗',
192:         '写给孩子',
193:         '致敬母亲',
194:         '送给朋友',
195:         '适合摘抄',
196:         '可以用作签名'
197:     ]
198: }
199: ];
200:
201: export default tagCategories;

```

==== FILE: utils/time.js =====

```

1: const pad = (num, len = 2) => String(num).padStart(len, '0');
2:
3: const formatDate = (input, pattern = 'yyyy-MM-dd HH:mm:ss') => {
4:     const d = input instanceof Date ? input : new Date(input);
5:     if (Number.isNaN(d.getTime())) return '';
6:     const map = {
7:         yyyy: String(d.getFullYear()),
8:         MM: pad(d.getMonth() + 1),
9:         dd: pad(d.getDate()),
10:        HH: pad(d.getHours()),
11:        mm: pad(d.getMinutes()),
12:        ss: pad(d.getSeconds())
13:    };
14:    return pattern.replace(/yyyy|MM|dd|HH|mm|ss/g, (t) => map[t]);
15: };
16:
17: const formatRelativeTime = (input) => {
18:     const now = Date.now();
19:     const t = input instanceof Date ? input.getTime() : new Date(input).getTime();
20:     if (Number.isNaN(t)) return '';
21:     const diffMs = now - t;
22:     const absSec = Math.floor(Math.abs(diffMs) / 1000);
23:     const dir = diffMs >= 0 ? '前' : '后';
24:     if (absSec < 60) return `刚刚`;
25:     const mins = Math.floor(absSec / 60);
26:     if (mins < 60) return `${mins}分钟${dir}`;
27:     const hours = Math.floor(mins / 60);
28:     if (hours < 24) return `${hours}小时${dir}`;
29:     const days = Math.floor(hours / 24);
30:     if (days < 7) return `${days}天${dir}`;

```

```

31:     return formatDate(t, 'yyyy-MM-dd');
32: };
33:
34: const formatTimeAgo = (timestamp) => {
35:     const now = new Date();
36:     const past = new Date(timestamp);
37:     const diff = now.getTime() - past.getTime();
38:     const seconds = Math.floor(diff / 1000);
39:     const minutes = Math.floor(seconds / 60);
40:     const hours = Math.floor(minutes / 60);
41:     const days = Math.floor(hours / 24);
42:     if (days >= 7) {
43:         const year = past.getFullYear();
44:         const month = ('0' + (past.getMonth() + 1)).slice(-2);
45:         const day = ('0' + past.getDate()).slice(-2);
46:         return `${year}-${month}-${day}`;
47:     } else if (days >= 1) {
48:         return `${days}天前`;
49:     } else if (hours >= 1) {
50:         return `${hours}小时前`;
51:     } else if (minutes >= 1) {
52:         return `${minutes}分钟前`;
53:     } else {
54:         return '刚刚';
55:     }
56: };
57:
58: module.exports = {
59:     formatTimeAgo,
60:     formatRelativeTime,
61:     formatDate
62: };

```

===== FILE: utils/timeline.js =====

```

1: /**
2:  * 时间轴相关工具函数
3:  */
4:
5: /**
6:  * 按月份分组帖子
7:  * @param {Array} posts - 帖子列表
8:  * @returns {Object} 按月份分组的帖子对象
9:  */
10: export function groupPostsByMonth(posts) {
11:     const groups = {};
12:     if (!Array.isArray(posts)) return groups;
13:
14:     posts.forEach(post => {
15:         if (!post.createTime) return;
16:

```

```

17:     try {
18:         const date = new Date(post.createTime);
19:         if (isNaN(date.getTime())) return;
20:
21:         const monthKey = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2, '0')}`;
22:         if (!groups[monthKey]) {
23:             groups[monthKey] = [];
24:         }
25:         groups[monthKey].push(post);
26:     } catch (error) {
27:         console.warn('【timeline】处理帖子时间失败:', error, post);
28:     }
29: });
30:
31: return groups;
32: }
33:
34: /**
35:  * 处理时间轴帖子的日期显示
36:  * @param {Array} posts - 帖子列表
37:  * @returns {Array} 处理后的帖子列表
38:  */
39: export function processPostsForTimeline(posts) {
40:     if (!Array.isArray(posts) || posts.length === 0) return [];
41:
42:     try {
43:         // 按日期排序（最新的在前）
44:         const sortedPosts = posts.sort((a, b) => {
45:             const timeA = new Date(a.createTime).getTime();
46:             const timeB = new Date(b.createTime).getTime();
47:             return timeB - timeA;
48:         });
49:
50:         // 处理同一天帖子的日期显示
51:         const processedPosts = [];
52:         let lastDate = null;
53:
54:         sortedPosts.forEach(post => {
55:             try {
56:                 const postDate = new Date(post.createTime);
57:                 if (isNaN(postDate.getTime())) return;
58:
59:                 const dateStr = `${postDate.getFullYear()}-${String(postDate.getMonth() + 1).padStart(2, '0')}-${String(postDate. ...
60:
61:                 // 如果是同一天，不显示日期
62:                 const showDate = lastDate !== dateStr;
63:                 lastDate = dateStr;
64:
65:                 processedPosts.push({
66:                     ...post,

```

```

67:         showDate: showDate,
68:         dateStr: dateStr
69:     });
70:     } catch (error) {
71:         console.warn('【timeline】处理单个帖子失败:', error, post);
72:     }
73: });
74:
75:     return processedPosts;
76: } catch (error) {
77:     console.error('【timeline】处理帖子列表失败:', error);
78:     return posts;
79: }
80: }
81:
82: /**
83:  * 格式化日期标签（只显示日子）
84:  * @param {string} dateKey - 日期字符串 (yyyy-MM-dd)
85:  * @returns {string} 格式化后的日期标签
86:  */
87: export function formatDateLabel(dateKey) {
88:     if (!dateKey || typeof dateKey !== 'string') return '';
89:
90:     try {
91:         const parts = dateKey.split('-');
92:         if (parts.length >= 3) {
93:             return `${parts[2]}日`;
94:         }
95:         return dateKey;
96:     } catch (error) {
97:         console.warn('【timeline】格式化日期标签失败:', error, dateKey);
98:         return dateKey;
99:     }
100: }
101:
102: /**
103:  * 格式化月份标签
104:  * @param {string} monthKey - 月份字符串 (yyyy-MM)
105:  * @returns {string} 格式化后的月份标签
106:  */
107: export function formatMonthLabel(monthKey) {
108:     if (!monthKey || typeof monthKey !== 'string') return '';
109:
110:     try {
111:         const parts = monthKey.split('-');
112:         if (parts.length >= 2) {
113:             const year = parts[0];
114:             const month = parseInt(parts[1], 10);
115:             return `${year}年${month}月`;
116:         }

```

```

117:     return monthKey;
118:   } catch (error) {
119:     console.warn('【timeline】格式化月份标签失败:', error, monthKey);
120:     return monthKey;
121:   }
122: }
123:
124: /**
125:  * 切换月份折叠状态
126:  * @param {Object} collapsedMonths - 当前折叠状态对象
127:  * @param {string} monthKey - 月份键
128:  * @returns {Object} 新的折叠状态对象
129:  */
130: export function toggleMonthCollapse(collapsedMonths, monthKey) {
131:   if (!monthKey || typeof monthKey !== 'string') {
132:     return collapsedMonths || {};
133:   }
134:
135:   const newCollapsed = { ...collapsedMonths || {} };
136:   if (newCollapsed[monthKey]) {
137:     delete newCollapsed[monthKey];
138:   } else {
139:     newCollapsed[monthKey] = true;
140:   }
141:
142:   return newCollapsed;
143: }

```

===== FILE: utils/uiHelpers.js =====

```

1: /**
2:  * UI相关的工具函数
3:  * 提供常用的UI操作和样式生成功能
4:  */
5:
6: /**
7:  * 生成随机背景色（避免连续重复）
8:  * @param {Array} colors - 可用的颜色数组
9:  * @param {number} lastUsedIndex - 上次使用的颜色索引
10:  * @returns {Object} 包含新颜色和索引的对象
11:  */
12: export function generateRandomBackgroundColor(colors, lastUsedIndex) {
13:   if (!colors || colors.length === 0) {
14:     return { color: '#f5f5f5', index: 0 };
15:   }
16:
17:   if (lastUsedIndex === -1) {
18:     const idx = Math.floor(Math.random() * colors.length);
19:     return { color: colors[idx], index: idx };
20:   }
21:

```

```

22:   const avail = colors.filter((_, i) => i !== lastUsedIndex);
23:   const pick = avail[Math.floor(Math.random() * avail.length)];
24:   const newIndex = colors.indexOf(pick);
25:   return { color: pick, index: newIndex };
26: }
27:
28: /**
29:  * 切换数组的展开状态
30:  * @param {Array} array - 要处理的数组
31:  * @param {number} index - 要切换的索引
32:  * @returns {Array} 新的数组
33:  */
34: export function toggleArrayItemExpansion(array, index) {
35:   if (!array || index < 0 || index >= array.length) {
36:     return array;
37:   }
38:
39:   const newArray = [...array];
40:   newArray[index] = {
41:     ...newArray[index],
42:     isExpanded: !newArray[index].isExpanded
43:   };
44:
45:   return newArray;
46: }
47:
48: /**
49:  * 从列表中提取可用的帖子ID
50:  * @param {Array} postList - 帖子列表
51:  * @returns {Array} 过滤后的ID数组
52:  */
53: export function extractValidPostIds(postList) {
54:   if (!Array.isArray(postList)) {
55:     return [];
56:   }
57:
58:   return postList
59:     .map(p => p._id)
60:     .filter(Boolean);
61: }
62:
63: /**
64:  * 过滤匿名帖子
65:  * @param {Array} postList - 帖子列表
66:  * @returns {Array} 过滤后的帖子列表
67:  */
68: export function filterAnonymousPosts(postList) {
69:   if (!Array.isArray(postList)) {
70:     return [];
71:   }

```

```

72:
73:   return postList.filter(p => p && !p.isAnonymous);
74: }
75:
76: /**
77:  * 批量更新帖子的UI属性
78:  * @param {Array} postList - 帖子列表
79:  * @param {Function} backgroundColorGenerator - 背景色生成函数
80:  * @param {Object} likeIconUtil - 点赞图标工具
81:  * @param {string} defaultTextColor - 默认文字颜色
82:  * @returns {Array} 更新后的帖子列表
83:  */
84: export function updatePostsUIProperties(postList, backgroundColorGenerator, likeIconUtil, defaultTextColor = '#222') {
85:   if (!Array.isArray(postList)) {
86:     return [];
87:   }
88:
89:   return postList.map((post, index) => {
90:     if (!post) return post;
91:
92:     // 生成背景色
93:     const bgResult = backgroundColorGenerator(post.backgroundColor);
94:
95:     return {
96:       ...post,
97:       backgroundColor: post.backgroundColor || bgResult.color,
98:       textColor: post.textColor || defaultTextColor,
99:       isExpanded: false,
100:      authorSignature: post.authorSignature || '',
101:      likeIcon: likeIconUtil && likeIconUtil.getLikeIcon
102:        ? likeIconUtil.getLikeIcon(post.votes || 0, !!post.isVoted)
103:        : ''
104:    };
105:  });
106: }
107:
108: /**
109:  * 合并帖子列表（避免重复）
110:  * @param {Array} existingList - 现有列表
111:  * @param {Array} newList - 新列表
112:  * @param {boolean} prepend - 是否将新列表放在前面（用于刷新）
113:  * @returns {Array} 合并后的列表
114:  */
115: export function mergePostLists(existingList, newList, prepend = false) {
116:   if (!Array.isArray(existingList) || !Array.isArray(newList)) {
117:     return [];
118:   }
119:
120:   const existingIds = new Set(existingList.map(p => p._id).filter(Boolean));
121:   const uniqueNewPosts = newList.filter(p => p && p._id && !existingIds.has(p._id));

```

```

122:
123:   if (prepend) {
124:     return [...uniqueNewPosts, ...existingList];
125:   } else {
126:     return [...existingList, ...uniqueNewPosts];
127:   }
128: }

```

===== FILE: utils/uploader.js =====

```

1: // utils/uploader.js
2: // 通用文件上传工具模块
3: // 提取自 profile-edit.vue, 支持多端兼容 (H5、App、小程序) 和自动回退机制
4:
5: const { cloudCall } = require('./cloudCall.js');
6: const { getCloudFunctionMethod, getCurrentPlatform } = require('./platformDetector.js');
7:
8: /**
9:  * 通过云函数中转上传文件 (作为备用方案)
10:  * @param {string} cloudPath - 云存储路径
11:  * @param {string} filePath - 本地文件临时路径
12:  * @param {number} [retryCount=0] - 当前重试次数
13:  * @returns {Promise<string>} - 返回文件FileID
14:  */
15: function uploadFileViaCloudFunction(cloudPath, filePath, retryCount = 0) {
16:   return new Promise((resolve, reject) => {
17:     const platform = getCurrentPlatform();
18:
19:     const performUpload = (base64) => {
20:       cloudCall("upload", { cloudPath, fileContent: base64 }, { pageTag: 'uploader', requireAuth: false })
21:         .then((uploadRes) => {
22:           if (uploadRes && uploadRes.result && uploadRes.result.success) {
23:             resolve(uploadRes.result.fileID);
24:           } else {
25:             reject(new Error("云函数返回异常"));
26:           }
27:         })
28:         .catch((err) => {
29:           // 简单的重试机制
30:           if (retryCount < 2) {
31:             setTimeout(() => {
32:               uploadFileViaCloudFunction(cloudPath, filePath, retryCount + 1)
33:                 .then(resolve).catch(reject);
34:             }, 1000 * (retryCount + 1));
35:           } else {
36:             reject(err);
37:           }
38:         });
39:   });
40:
41:   if (platform === 'h5') {

```



```

42: // H5环境: 使用fetch获取blob, 然后转换为base64
43: console.log('□ [Uploader] H5环境使用fetch读取文件');
44: fetch(filePath)
45:   .then(response => {
46:     if (!response.ok) throw new Error("HTTP " + response.status);
47:     return response.blob();
48:   })
49:   .then(blob => {
50:     const reader = new FileReader();
51:     reader.onload = () => {
52:       const result = reader.result;
53:       if (!result || typeof result !== "string") {
54:         reject(new Error("文件读取失败"));
55:         return;
56:       }
57:       const base64 = result.split(",")[1];
58:       console.log(`□ [Uploader] H5环境文件读取完成, base64长度: ${base64.length}`);
59:       performUpload(base64);
60:     };
61:     reader.onerror = () => reject(new Error("文件读取失败"));
62:     reader.readAsDataURL(blob);
63:   })
64:   .catch(err => reject(err));
65:
66: } else if (platform === 'app') {
67:   // App环境使用 plus.io 读取为 base64
68:   console.log('□ [Uploader] App环境使用plus.io读取文件');
69:   if (typeof plus === 'undefined' || !plus.io) {
70:     reject(new Error('App端运行环境不可用'));
71:     return;
72:   }
73:   plus.io.resolveLocalFileSystemURL(filePath, (entry) => {
74:     entry.file((file) => {
75:       const reader = new plus.io.FileReader();
76:       reader.onload = (e) => {
77:         try {
78:           const dataUrl = e.target.result || '';
79:           const base64 = dataUrl.includes(',') ? dataUrl.split(',')[1] : dataUrl;
80:           if (!base64) {
81:             reject(new Error('文件读取失败或为空'));
82:             return;
83:           }
84:           performUpload(base64);
85:         } catch (ex) {
86:           reject(new Error('读取转换失败: ' + ex.message));
87:         }
88:       };
89:       reader.onerror = (err) => reject(new Error('文件读取失败: ' + (err && err.message || 'unknown')));
90:       reader.readAsDataURL(file);
91:     }, (e) => reject(new Error('获取文件对象失败: ' + (e && e.message || 'unknown'))));

```

```

92:         }, (e) => reject(new Error('路径解析失败: ' + (e && e.message || 'unknown'))));
93:
94:     } else {
95:         // 小程序等其他平台: 使用 uni.getFileSystemManager
96:         console.log('☐ [Uploader] 小程序环境使用uni.getFileSystemManager读取文件');
97:         const fs = uni.getFileSystemManager();
98:         if (!fs || !fs.readFile) {
99:             reject(new Error('文件系统管理器不可用'));
100:            return;
101:        }
102:        fs.readFile({
103:            filePath: filePath,
104:            encoding: 'base64',
105:            success: (readRes) => {
106:                const base64 = readRes.data;
107:                console.log('☐ [Uploader] 小程序环境文件读取完成, base64长度: ${base64.length}`);
108:                performUpload(base64);
109:            },
110:            fail: (readErr) => {
111:                console.error('☐ [Uploader] 小程序环境文件读取失败:', readErr);
112:                reject(new Error('文件读取失败: ' + readErr.errMsg));
113:            }
114:        });
115:    }
116: });
117: }
118:
119: /**
120:  * 通用文件上传函数 (主入口)
121:  * 优先尝试客户端直传, 失败后自动回退到通过云函数中转。
122:  * @param {string} cloudPath - 云存储路径
123:  * @param {string} filePath - 本地文件临时路径
124:  * @returns {Promise<string>} - 返回文件FileID
125:  */
126: async function uploadFile(cloudPath, filePath) {
127:     const method = getCloudFunctionMethod();
128:
129:     if (method === 'tcb') {
130:         const app = getApp();
131:         if (!(app && app.$tcb && typeof app.$tcb.uploadFile === 'function')) {
132:             console.warn('[Uploader] TCB实例不可用, 回退到云函数上传');
133:             return await uploadFileViaCloudFunction(cloudPath, filePath);
134:         }
135:         let file = filePath;
136:         try {
137:             // H5/App环境, 尝试将路径转为Blob对象直传, 性能更优
138:             if (typeof filePath === "string" && typeof fetch === "function" && typeof Blob !== "undefined") {
139:                 const resp = await fetch(filePath);
140:                 file = await resp.blob();
141:             } else if (filePath && filePath.tempFilePath && typeof fetch === "function") {

```

```

142:         const resp = await fetch(filePath.tempFilePath);
143:         file = await resp.blob();
144:     }
145: } catch (e) {
146:     console.warn('[Uploader] toBlob失败, 改走云函数上传', e);
147:     return await uploadFileViaCloudFunction(cloudPath, filePath);
148: }
149: try {
150:     const res = await app.$tcb.uploadFile({ cloudPath, file });
151:     const fileId = (res && (res.fileID || res.fileid)) || (res && res.data && res.data.fileID);
152:     if (!fileID) {
153:         throw new Error('TCB直传成功但未返回fileID');
154:     }
155:     return fileId;
156: } catch (e) {
157:     console.warn('[Uploader] TCB直传失败 (${e.message}), 回退到云函数上传...');
158:     return await uploadFileViaCloudFunction(cloudPath, filePath);
159: }
160:
161: } else if (method === "wx-cloud") {
162:     try {
163:         const res = await wx.cloud.uploadFile({ cloudPath, filePath });
164:         const fileId = res && res.fileID;
165:         if (!fileID) {
166:             throw new Error('小程序上传成功但未返回fileID');
167:         }
168:         return fileId;
169:     } catch (e) {
170:         console.warn('[Uploader] 小程序直传失败 (${e.message}), 回退到云函数上传...');
171:         return await uploadFileViaCloudFunction(cloudPath, filePath);
172:     }
173: }
174:
175: // 如果平台检测不到, 直接使用云函数中转作为默认方案
176: console.log('[Uploader] 未知上传环境, 默认使用云函数上传');
177: return await uploadFileViaCloudFunction(cloudPath, filePath);
178: }
179:
180: module.exports = {
181:     uploadFile
182: };
183:

```

===== FILE: utils/viewEvents.js =====

```

1: const { cloudCall } = require('./cloudCall.js');
2:
3: let queue = [];
4: let started = false;
5: let timer = null;
6:

```

```

7: function getSessionId() {
8:   try {
9:     if (typeof uni !== 'undefined' && uni.getStorageSync) {
10:       let sid = uni.getStorageSync('sessionId');
11:       if (!sid) {
12:         sid = Math.random().toString(36).slice(2) + Date.now().toString(36);
13:         uni.setStorageSync('sessionId', sid);
14:       }
15:       return sid;
16:     }
17:   } catch (e) {}
18:   return 's';
19: }
20:
21: function enqueueView(postId, duration) {
22:   if (!postId) return;
23:   const now = Date.now();
24:   queue.push({ postId, ts: now, sessionId: getSessionId(), duration: Number(duration || 0) || 0 });
25:   // 懒启动自动 flush
26:   if (!started) start();
27: }
28:
29: async function flush() {
30:   if (!queue.length) return { success: true, count: 0 };
31:   const batch = queue.splice(0, 50);
32:   try {
33:     const res = await cloudCall('recordEventBatch', { events: batch }, { pageTag: 'viewEvents', retry: 1 });
34:     return res && res.result ? res.result : { success: true, count: batch.length };
35:   } catch (e) {
36:     // 失败回退：合并回队列前端重试
37:     queue = batch.concat(queue);
38:     return { success: false, error: e };
39:   }
40: }
41:
42: function start() {
43:   started = true;
44:   if (typeof setInterval === 'function' && !timer) {
45:     timer = setInterval(() => { flush(); }, 10000);
46:   }
47:   try { if (typeof uni !== 'undefined' && typeof uni.onHide === 'function') { uni.onHide(() => { flush(); }); } } catch (...) {}
48: }
49:
50: module.exports = {
51:   enqueueView,
52:   flushViewQueue: flush,
53:   startViewEventAutoFlush: start
54: };
55:

```